FAKULTA MECHATRONIKY,
INFORMATIKY A MEZIOBOROVÝCH
STUDIÍ TUL

Diplomová práce

Multiplatform Desktop Application for Visualization and Analysis of Telemetry Data from Automotive Test Drives

Studijní program: N0613A140028 Informační technologie
Autor práce: **Bc. Petr Boháč**
Vedoucí práce: Ing. Jan Kolaja, Ph.D.
Ústav nových technologií a aplikované informatiky

Liberec 2026

FAKULTA MECHATRONIKY,
INFORMATIKY A MEZIOBOROVÝCH
STUDIÍ TUL

Zadání diplomové práce

Multiplatform Desktop Application for Visualization and Analysis of Telemetry Data from Automotive Test Drives

Jméno a příjmení:	**Bc. Petr Boháč**
Osobní číslo:	M24000132
Studijní program:	N0613A140028 Informační technologie
Zadávající katedra:	Ústav nových technologií a aplikované informatiky
Akademický rok:	2025/2026

Zásady pro vypracování:

1. Conduct a thorough research on existing solutions and open projects for processing, visualizing, and analyzing telemetry data from cars, including data formats, protocols, and visualization methods.
2. Design the architecture of a multi-platform desktop application for managing and visualizing telemetry data, describing the technologies, application structure, and data model.
3. Implement visualization tools that allow the selection of data channels, the combination of data from different trips or vehicles, and display in graphical and statistical form.
4. Extend the application with analytical functions, including basic statistics and selected machine learning techniques for telemetry analysis.
5. Consider options for further expanding the solution in terms of functionality, scalability, and integration into the car manufacturer's development chain.
6. Prepare testing and documentation, ensure that functional and user tests are carried out in cooperation with car manufacturer employees, and properly comment on the source code.

Rozsah grafických prací:	dle potřeby dokumentace
Rozsah pracovní zprávy:	40 až 50 stran
Forma zpracování práce:	tištěná/elektronická
Jazyk práce:	angličtina

Seznam odborné literatury:

1. MCKINNEY, Wes. Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter. 3rd ed. Beijing: O'Reilly Media, 2022. ISBN 978-1-098-10774-1.
2. VANDERPLAS, Jake. Python Data Science Handbook: Essential Tools for Working with Data. 2nd ed. Sebastopol: O'Reilly Media, 2022. ISBN 978-1-098-11931-7.
3. MATPLOTLIB DEVELOPMENT TEAM. Matplotlib: Visualization with Python [online]. 2023. Dostupné z: https://matplotlib.org/stable/
4. SCIKIT-LEARN DEVELOPERS. scikit-learn: Machine Learning in Python [online]. 2023. Dostupné z: https://scikit-learn.org/

Vedoucí práce:	Ing. Jan Kolaja, Ph.D. Ústav nových technologií a aplikované informatiky
Datum zadání práce:	17. prosince 2025
Předpokládaný termín odevzdání:	11. května 2026

L.S.

doc. Ing. Josef Černohorský, Ph.D.
děkan

doc. Ing. Petr Červa, Ph.D.
garant studijního programu

V Liberci dne 17. prosince 2025

Prohlášení

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.

Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.

Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.

Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitu v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.

Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS/STAG se shoduje s textem tištěné podoby práce.

Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.

Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

6. května 2026 Bc. Petr Boháč

Multiplatformní desktopová aplikace pro vizualizaci a analýzu telemetrických dat z testovacích jízd automobilů

ABSTRAKT

Tato diplomová práce se zabývá následným zpracováním telemetrických dat z testovacích jízd automobilů ve společnosti Škoda Auto. Cílem bylo analyzovat současný pracovní postup inženýrů a techniků, zhodnotit existující softwarová řešení a navrhnout a implementovat multiplatformní desktopovou aplikaci, která zjednoduší vizualizaci a analýzu exportovaných měřicích dat. Analytická část identifikuje hlavní slabiny stávajícího workflow, zejména opakované ruční zpracování CSV exportů, obtížné porovnávání více měření a absenci lehkého nástroje přizpůsobeného každodennímu inženýrskému použití. Na základě průzkumu existujících nástrojů byla navržena specializovaná aplikace využívající jazyk Python a UI framework PyQt. Implementované řešení podporuje načítání a předzpracování dat exportovaných z ODIS, normalizaci kanálů, práci s více datovými sadami, flexibilní vykreslování grafů, deskriptivní statistiku, vizualizaci rozdílových křivek, korelační analýzu, detekci anomálií založenou na směrodatné odchylce a rozšíření o metody strojového učení bez učitele. Práce se dále věnuje testování, dokumentaci, verzování a zabalení aplikace pro nasazení v restriktivním podnikovém prostředí. Výsledkem je funkční softwarový nástroj, který urychluje rutinní analýzu dat z testovacích jízd a vytváří základ pro další rozšiřování funkcionality.

Klíčová slova: telemetrická data, testovací jízdy automobilů, desktopová aplikace, vizualizace dat, detekce anomálií, Python, PyQt

Multiplatform desktop application for visualization and analysis of telemetry data from car test drives

ABSTRACT

This thesis deals with the post-processing of telemetry data from automotive test drives at Skoda Auto. The goal was to analyze the current workflow of engineers and technicians, evaluate existing software solutions, and design and implement a multiplatform desktop application that simplifies the visualization and analysis of exported measurement data. The analytical part identifies the main weaknesses of the existing workflow, especially repetitive manual processing of CSV exports, difficult comparison of multiple measurements, and the absence of a lightweight tool adapted to everyday engineering use. Based on the survey of existing tools, a dedicated application using Python and PyQt for UI was proposed. The implemented solution supports loading and preprocessing ODIS-exported data, normalization of channels, work with multiple datasets, flexible plotting, descriptive statistics, delta-curve visualization, correlation analysis, standard deviation-based anomaly detection, and an unsupervised machine-learning extension. The thesis also covers testing, documentation, versioning, and packaging of the application for deployment in a restrictive corporate environment. The result is a working software tool that accelerates the routine analysis of test-drive data and provides a foundation for future functional expansion.

Keywords: telemetry data, automotive test drives, desktop application, data visualization, anomaly detection, Python, PyQt

ACKNOWLEDGEMENT

I would like to acknowledge everyone who contributed to the creation of this fine piece of work.

Bc. Petr Boháč

CONTENTS

LIST OF TABLES

LIST OF IMAGES

LIST OF ABBREVIATIONS

OBD-II	On-Board Diagnostics II
ECU	Electronic Control Unit
ODIS	Offboard Diagnostic Information System
PWM	Pulse-Width Modulation
INCA	Integrated Calibration and Acquisition System
MDF	Measurement Data Format
IDE	Integrated Development Environment
UI	User Interface
UX	User Experience
PFP	Per-File Plot
PCP	Per-Channel Plot
ML	Machine Learning
QoL	Quality of Life
OS	Operating System
AI	Artificial Intelligence

1 INTRODUCTION

The development of modern passenger vehicles is an iterative engineering process in which assumptions made during design must be repeatedly verified under real operating conditions. Although simulations, laboratory measurements, and component-level tests provide valuable insight, they cannot fully capture the complexity of vehicle behavior in practice. For that reason, automotive manufacturers continue to rely on test drives and other experimental procedures in which a broad range of operating variables is recorded and evaluated. These measurements form a rich source of telemetry data describing the state of the vehicle over time and across varying environmental conditions.

This context is highly relevant at Škoda Auto, where vehicle development involves repeated validation of individual subsystems as well as the overall behavior of the car. In the Thermosystem department, where the practical part of this thesis is situated, engineers and technicians work with data obtained during standardized test drives, diagnostic measurements, and other verification activities.[1] As the number of tests grows, so does the volume of exported measurement data that must be inspected, compared, and interpreted. The ability to process this data efficiently therefore has a direct impact on the speed and quality of engineering decision-making.

The challenge addressed in this thesis does not lie primarily in the acquisition of data, but in its subsequent post-processing. In the examined workflow, engineers often work with tabular exports that need to be manually cleaned, compared across multiple runs, and transformed into graphs or summary statistics before they become useful for technical interpretation. This approach is workable, but it is also repetitive, time-consuming, and poorly suited to routine comparison of larger numbers of measurements. Existing enterprise platforms can cover parts of this process, but they are often designed for different use cases, require specialized ecosystems, or do not provide the lightweight comparative workflow needed in everyday practice. This creates room for a focused software tool tailored to the needs of the target users.

The main objective of this thesis is to design and implement a multiplatform desktop application for the visualization and analysis of telemetry data from automotive test drives. The application is intended to support engineers and technicians during routine post-processing of exported measurements by mak-

[1] The concrete workflow described in this thesis reflects the environment of the Thermosystem department at Škoda Auto, although many of the underlying principles are applicable more broadly.

ing common analytical tasks faster, clearer, and less dependent on ad hoc manual work.

2 PROBLEM ANALYSIS

As previously mentioned, the development process for a car generates a large amount of telemetry data that engineers and technicians must analyze. The current workflow in our department for analyzing this data is inefficient and time-consuming, underscoring the need for a software solution to streamline the process.

2.1 MOTIVATION

To illustrate the practical motivation behind telemetry analysis, consider a situation in which a vehicle exhibits undesirable behavior only under specific operating conditions, for example, insufficient cabin heating during winter driving, unexpectedly high coolant temperature during a sustained uphill climb, or unstable behavior of a controlled component such as a radiator fan. In such cases, the problem cannot usually be reliably diagnosed from a single subjective observation. What matters is not only that the undesired behavior occurred, but also **when** it occurred, under what load, at what speed, at what ambient temperature, and in relation to which other measured variables.

This is precisely why test drives and measurement campaigns are essential to vehicle development. By recording telemetry data under controlled or repeatable conditions, engineers gain insight into the technical context surrounding a problem. They can reconstruct the sequence of events, compare multiple runs, observe interactions between signals, and verify whether the measured data actually support a suspected cause. Telemetry, therefore, serves not merely as an archival by-product of testing but as a primary source of evidence for technical reasoning, validation, and decision-making.

At the same time, the usefulness of telemetry data depends heavily on how easily it can be explored after acquisition. Even a well-designed test loses value if the resulting measurements are difficult to compare, slow to visualize, or cumbersome to interpret. The motivation for this thesis, therefore, lies not only in the existence of measurement data but in the need to transform that data into a form that supports efficient day-to-day analytical work. For that reason, the following sections first describe how the relevant data is collected and then examine the current workflow used for its analysis.

2.2 DATA COLLECTION

Because different conditions reveal different aspects of a car's performance, a single standardized test is not sufficient. Instead, cars must be tested across a spectrum of environments. These environments include, but are not limited to, public roads, test tracks, and wind tunnels. By adapting each test to a specific area of investigation, engineers can gain deeper insights into how a car behaves under particular conditions, which is crucial for optimizing its design and performance. A small selection of tests used in production is illustrated in Figure 2. 1, taken from the internal configuration system, also developed by the author.

Název	Identifikátor	Prostředí	$T_{W\_HWK\_ein}$ (°C)	$T_{L\_AUSSEN\_s}$ (°C)	$T_{L\_AUSSEN\_e}$ (°C)	ETD_s (°C)	ETD_e (°C)	T_{o-mot} (°C)	ATD (°C)	Rychlost (km/h)	Náklon (%)
6% stoupání 45°C	*6% stoupání 45°C*	Tunel	108,00	45,00	45,00	63,00	63,00	150,00	-	-	6,00
8% stoupání 45°C	*8% stoupání 45°C*	Tunel	108,00	45,00	45,00	63,00	63,00	150,00	-	-	8,00
FT mit Anhänger	*FTmit*	Tunel	108,00	24,00	24,00	84,00	84,00	150,00	-	70,00	9,00
FT mit Anhänger REAL	*Ftmit*	Silnice	117,00	30,00	24,00	87,00	93,00	150,00	-	70,00	-
GG mit Anhänger	*GGm*	Tunel	108,00	30,00	18,00	78,00	90,00	150,00	50,00	30,00	12,00
GG mit Anhänger REAL	*GGm*	Silnice	117,00	30,00	18,00	87,00	99,00	150,00	50,00	30,00	-
GG ohne Anhänger	*GGoA*	Tunel	108,00	30,00	18,00	78,00	90,00	150,00	40,00	55,00	12,00

Figure 2. 1: A snippet of the predefined tests with various parameters.

In general, vehicle telemetry data can be collected in two primary ways: through the integrated On-Board Diagnostics II (OBD-II) port [1], or by equipping the vehicle with a large number of additional sensors while simultaneously aggregating data from the car's internal systems.

2.2.1 OBD-II-BASED DATA ACQUISITION

The former method is significantly more straightforward and is commonly used for basic diagnostics and performance monitoring. The setup typically involves connecting a data-logging device, most often a laptop computer, to the vehicle's OBD-II port using a standard interface cable [1]. The computer then runs software that can communicate with the vehicle's onboard systems, allowing it to read and record various parameters from the car's Electronic Control Unit (ECU) [2].

Any software capable of reading data from the aforementioned port can be used for this purpose; many options are available, both open source and commercial. At Škoda Auto, part of the Volkswagen Group, the software commonly used for this task is Offboard Diagnostic Information System (ODIS), a Volkswagen Group diagnostic system (see Figure 2. 2). It is licensed to authorized users and

provides a comprehensive set of diagnostic functions for vehicle development and service use [3].

Figure 2. 2: ODIS Factory Diagnostic OEM Pro Package used in Škoda Auto. Source: https://diagnoex.com/products/vw-audi-odis-scan-tool

The output is typically exported in CSV format, which can be easily imported into spreadsheet software or other data analysis tools for further processing. This method is trivial to set up and can provide a wide range of information about the vehicle's operation, including engine speed, coolant temperature, throttle position, and other parameters. However, due to limitations in available signals and sampling rates, it may not capture all the data required for more detailed or high-fidelity analysis.

2.2.2 INSTRUMENTED VEHICLE DATA ACQUISITION

The second method of data collection involves equipping the vehicle with a large number of additional sensors and aggregating data from the vehicle's internal systems using dedicated supporting hardware (similar to Figure 2. 3). In practice, this means lifting the vehicle, disassembling parts of the engine bay, and installing various sensors to measure parameters not available through the OBD-II port. This can include additional temperature, pressure, and accelerometer sensors, as well as other specialized equipment, depending on the specific requirements of the test. An example of such a setup is shown in Attachment 2, where various sensors are installed in the engine bay of a test vehicle. The data from these sensors, along with signals from the car's internal systems, is

then aggregated using dedicated hardware, such as the ETAS data acquisition system (as shown in Attachments 4 and 5), which can handle high sampling rates and large volumes of data.

Compared to the first approach, this method is significantly more complex and costly, and the installation process typically requires approximately two days. However, it enables the collection of far more comprehensive and detailed telemetry data, which is essential for in-depth analysis and optimization of vehicle performance.

Figure 2. 3: ETAS Hardware for Data Acquisition
Source: https://www.etas.com/ww/en/products-services/data-acquisition-processing-tools

For example, the ECU often regulates certain components through feedback control loops rather than providing direct measurements. A typical case is the main radiator fan, whose operation is controlled via Pulse-Width Modulation (PWM) to adjust cooling power dynamically. As a result, the fan speed does not directly correspond to the engine speed, and the actual fan rotational speed is not always available as a measured signal. In such cases, the fan speed may need to be estimated indirectly from related variables such as voltage, current, and PWM duty cycle. By installing an additional sensor that directly measures the fan's rotational speed, the system can provide a precise, readily usable measurement, significantly simplifying subsequent analysis.

In this configuration, the logging device is typically a laptop computer running specialized third-party software designed for high-fidelity data acquisition (as illustrated in Attachment 3). At Škoda Auto, the software commonly used for this purpose is Integrated Calibration and Acquisition System (INCA), a commercial tool developed by ETAS GmbH. This software provides advanced capabilities for

data acquisition, calibration, and analysis, making it well-suited for the complex requirements of high-resolution telemetry data collection [4].

Compared to the OBD-II-based method, this approach generates significantly larger datasets, often containing measurements from thousands of sensors at different high sampling rates. A simple text-based format would be impractical for storing such large volumes of data, so a more sophisticated, standardized format is used. The most common format for this type of data is the Measurement Data Format (MDF), a binary file format designed to store large volumes of measurement data efficiently. The MDF format allows for the storage of complex datasets with multiple channels, varying sampling rates, and metadata, making it ideal for high-fidelity telemetry data [5]. The majority of technicians and engineers at Škoda Auto are working with version 3 of the MDF format, but slow adoption of version 4 is expected in the near future.

Because this method has had so much time to mature, the software and hardware ecosystem around it is well developed, and the process of collecting and analyzing data is relatively streamlined. This leaves the first method slightly neglected, motivating the development of a software solution that improves the workflow for OBD-II-based data acquisition and analysis.

2.3 CURRENT WORKFLOW

It is no secret that large corporations across industries prefer commercial software solutions for their operations, and the automotive industry is no exception. Microsoft is the dominant provider of office software, and many companies rely on it for almost all of their day-to-day operations, from email, meetings, and document editing to data analysis and visualization.

Microsoft Excel is an excellent tool for many tasks and can be more than sufficient for even complex data analysis. That is why it is so widely used in the industry, including at Škoda Auto, where it is the primary tool for analyzing telemetry data collected through the OBD-II port. The current workflow is as follows:

2.3.1 DATA ACQUISITION

Data is collected using ODIS and exported in CSV format. The CSV data is then imported into Excel using its built-in import functionality, which allows users to specify data types and delimiters. Once imported, the Excel spreadsheet with a structure similar to that shown in Figure 2. 4 is saved to a shared network drive, making it accessible to other team members for further analysis and collaboration.

	A	B	C	D	E	F	G	H	I
1	#2026-03-03 12:56:41 TMBNH9NY0SF000226 trasa MB-MIM-MB, polojasno, 6st., test sady kanalu								
2	#Time	08/vnější te	01/Rychlos	01/buzení \	01/roleta ch	08/vysokon	08/vysokon	08/vysokon	08/vysokon
3	#Time	08----	01----	01----	01----	08-Timeout	08-stav top	08-Napájec	08-stav top
4	#ms	°C	km/h	%	%	-	-	-	-
5	4980	12	2	25.39	29	v poř.	v poř.	v poř.	v poř.
6	9856	11.5	5	25.39	29	v poř.	v poř.	v poř.	v poř.
7	15123	11	13	0	29	v poř.	v poř.	v poř.	v poř.
8	19879	10.5	14	25.39	29	v poř.	v poř.	v poř.	v poř.
9	24139	10	8	25.39	29	v poř.	v poř.	v poř.	v poř.
10	29220	9.5	12	25.39	29	v poř.	v poř.	v poř.	v poř.
11	34075	9	6	25.39	29	v poř.	v poř.	v poř.	v poř.
12	39060	8.5	7	25.39	29	v poř.	v poř.	v poř.	v poř.

Figure 2. 4: Structure of the Excel spreadsheet after importing the CSV data.

2.3.2 DATA ANALYSIS

Engineers and technicians use Excel's functions, formulas, and pivot tables to analyze the data. This analysis may include calculating summary statistics, creating charts and graphs, and performing basic data manipulation on a single file. When multiple test runs need to be compared, engineers must manually open several spreadsheets and compare the data side by side. This process can become cumbersome and error-prone as the number of files increases, especially on corporate-issued laptops, which often have limited processing power, memory, and a small screen.

It is this second part of the workflow that is particularly inefficient and time-consuming, which motivates the search for a better solution.

2.4 EXISTING SOLUTIONS

Before developing a custom tool, it is necessary to examine existing solutions for vehicle telemetry acquisition and analysis. These solutions range from professional automotive engineering tools used by manufacturers to more general-purpose diagnostic or telemetry platforms. While many of these tools provide powerful capabilities, they often target different use cases or impose limitations that make them unsuitable for the workflow described in this work.

2.4.1 PROFESSIONAL AUTOMOTIVE ENGINEERING TOOLS

In the automotive industry, several specialized software platforms exist for high-fidelity data acquisition, calibration, and analysis. These tools are primarily designed for original equipment manufacturers (OEMs) and their suppliers.

One widely used solution is CANoe, developed by Vector Informatik. The software is used for the development, analysis, and testing of electronic control units and distributed networks [6].

Another related tool from the same company is CANape, which focuses on the measurement and calibration of ECU parameters in real time. During measurement, engineers can adjust parameters while recording signals, with communication commonly handled via protocols such as XCP [7].

These tools provide extremely powerful capabilities but are typically expensive, require specialized hardware, and are primarily designed for ECU development rather than lightweight analysis of diagnostic telemetry data.

2.4.2 DIAGNOSTIC AND OBD-BASED TOOLS

Another category consists of tools designed for vehicle diagnostics using the OBD-II interface.

A well-known example is VCDS, developed by Ross-Tech. It is a diagnostic system for Volkswagen Group vehicles (VW, Audi, Škoda, and SEAT) [8].

Although such tools allow access to a wide range of vehicle parameters, their primary purpose is vehicle diagnostics rather than large-scale telemetry analysis. Data export and advanced analytical capabilities are typically limited compared to dedicated analysis platforms.

Some modern solutions attempt to extend OBD-based data analysis into broader telemetry applications. For example, platforms such as OBDAssistant provide real-time monitoring with hundreds of live parameters and AI-assisted interpretation of diagnostic information [9]. However, they are primarily targeted at consumer diagnostics and fleet monitoring rather than engineering analysis workflows.

2.4.3 TELEMETRY AND DATA ANALYSIS PLATFORMS

Several solutions exist that focus on telemetry visualization and analysis rather than diagnostics.

For instance, the AutoPi platform provides hardware and software solutions for collecting data from CAN, CAN-FD, and OBD-II networks and transmitting it to cloud infrastructure for fleet monitoring and analytics [10].

Similarly, specialized telemetry analysis portals are used in industrial contexts to store and analyze machine data over long periods, enabling users to run queries, generate reports, and define operational thresholds. These systems often integrate data from multiple hardware platforms and provide web-based dashboards for long-term analysis.

Another example is asammdf, an open-source tool and Python library designed for working with measurement data stored in the MDF format. It provides functionality for reading, writing, and processing MDF files and includes a graphical user interface for browsing signals, plotting telemetry data, and exporting measurements to other formats [11]. Because MDF is widely used in automotive testing environments, asammdf has become a popular tool for engineers and researchers working with recorded measurement datasets. In recent years, the tool has also begun to be adopted in practice at Škoda Auto, where engineers are gradually using it as an auxiliary tool for working with measurement data. However, it primarily focuses on processing MDF files rather than simplifying workflows built around CSV-based diagnostic data.

2.4.4 CONCLUSION ON EXISTING SOLUTIONS

Although the tools described above provide powerful capabilities, several limitations remain with respect to the workflow discussed in this work. Many professional engineering solutions require expensive licenses and specialized hardware. Although large corporations may have access to such resources, including Škoda Auto, the scope of the user base for which the results of this thesis are intended is limited to a single department, and the cost of deploying such tools across the entire user base would be prohibitive.

In addition, most existing tools are designed primarily for ECU development, fleet monitoring, or vehicle diagnostics rather than for the post-processing and comparative analysis of telemetry data exported from OBD-II systems. As a result, working with multiple test runs often requires manual processing across several files, typically using external tools such as spreadsheet software.

Furthermore, many professional platforms rely on proprietary data formats or tightly integrated ecosystems, which complicates the use of simple CSV-based datasets commonly produced during diagnostic measurements. These limitations motivate the development of a lightweight analysis tool tailored to OBD-II telemetry data to improve the efficiency and usability of the current workflow.

3 Designing the Software Architecture

The topic of multiplatform desktop applications is very deep, and it is easy to get caught up in the details of the various frameworks and technologies available for building them. That is why we need to establish some facts and the particular requirements of our application before we can start evaluating the options.

Pretty much all the computers used by non-IT professionals at Škoda Auto run Microsoft Windows, so we can safely assume that the target platform for our application is Windows. However, it is still desirable to run the application on other platforms, such as Linux or macOS, which underscores the need for a cross-platform solution.

3.1 Selecting the Programming Language

As for the programming language and its ecosystem, making the right choice is crucial to the project's long-term success, as it reduces the risk of insurmountable technical issues during development and maintenance. There are realistically two main approaches: native development using platform-specific tools and languages, or using a cross-platform framework that abstracts away platform-specific details and allows us to write code once and run it on multiple platforms.

Native applications, often written in languages such as C++ or C#, tend to have better performance and can take full advantage of platform-specific features. In particular, C++ is widely used for high-performance desktop applications and offers fine-grained control over system resources, which could make the application more efficient and snappier from a user perspective. However, native development typically requires separate codebases for each platform, which can be costly and time-consuming to develop and maintain.

On the other hand, cross-platform frameworks allow us to write a single codebase that runs on multiple platforms, significantly reducing development time and costs. One popular approach is to use JavaScript-based frameworks such as Electron, which enable developers to build desktop applications using web technologies like HTML, CSS, and JavaScript [12]. While Electron simplifies cross-platform development and has a large ecosystem, it is often associated

with higher memory usage and performance overhead compared to native solutions.

The main technical requirements for this project were rapid prototyping, ease of development, and maintainability for the author and possibly other developers in the future. C# with the .NET framework is a popular choice for Windows development, but its cross-platform capabilities, while improving, may still introduce limitations depending on the use case. Java is another option that is inherently cross-platform and has a large ecosystem, but it may be unnecessarily complex for a relatively simple desktop application.

Python, on the other hand, is a versatile language with a large ecosystem of libraries and frameworks for building desktop applications, and it has good support for cross-platform development. It is also widely used for data analysis and visualization, which makes it particularly suitable for applications that require such functionality [13, 14].

Many factors influenced the choice of Python for this project (as summarized in table Table 3. 1), but the most important were my familiarity with the language and its ecosystem, as well as its suitability for rapid prototyping and development. The ease of development and maintainability were also important considerations, as the application is intended for engineers and technicians who may not have extensive programming experience. Python's simplicity and readability make it an excellent choice for this purpose, enabling developers to understand and modify the codebase as needed quickly.

Table 3. 1: Comparison of programming languages for desktop application development

Language	Pros	Cons
C++	• High performance • Fine-grained control over system resources • Widely used for high-performance desktop applications	• Requires separate codebases for each platform • Longer development time • Steeper learning curve for non-C++ developers
C# with .NET	• Good performance • Strong support for Windows development • Improving cross-platform capabilities	• May still have limitations on non-Windows platforms • Potential overhead from the .NET runtime
Java	• Inherently cross-platform • Large ecosystem • Good performance	• May be unnecessarily complex for simple applications • Higher memory usage compared to native solutions
Python	• Versatile language with a large ecosystem of libraries and frameworks for desktop applications and data analysis • Good support for cross-platform development	• Generally slower than compiled languages like C++ or Java • May require additional tools for packaging and distribution

3.2 DESKTOP APPLICATION FRAMEWORKS

Software frameworks provide a structured way to build applications by offering pre-built components and tools that handle common tasks, such as user interface design, event handling, and data management. For desktop applications, there are several popular frameworks available for Python, including PyQt, Tkinter and Kivy.

Python is quite often used for building tools with simple graphical user interfaces using lightweight frameworks such as Pygame or the aforementioned Tkinter. Frameworks like these are easy to learn and can be sufficient for basic applications, but they may not provide the level of polish and user experience expected in a professional setting. For a more modern and responsive user interface, frameworks like PyQt or Kivy are often preferred. PyQt is a set of Python bindings for the Qt application framework, which is widely used for developing cross-platform applications with native look and feel. Kivy, on the other hand, is an open-source Python library for developing multitouch applications, which can

also be used for desktop applications. Both frameworks have their own strengths and weaknesses, so the choice between them came down to personal preference of the author.

PyQt has a slightly steeper learning curve and can be more complex to set up, but it offers a more traditional desktop application experience with a wide range of widgets and tools for building complex user interfaces. One particularly powerful feature of PyQt is its `QSettings` system, which provides a convenient platform-independant way to store and retrieve application settings across sessions. For example, on Windows, settings are typically stored in the registry, while on Linux they are stored in configuration files. By using QSettings' simple API (as shown in Listing 1), developers can abstract away these platform-specific details and ensure that user preferences and application state are preserved seamlessly across different operating systems.

```
settings = QSettings()

# Set a value
settings.setValue("username", "JohnDoe")

# Get a value
username = settings.value("username", "defaultUser")
```

Listing 1: Example of the QSettings API

The Qt framework (and consequently the PyQt library) has more than 25 years of development and a large community, which means that it is well-documented and has a wide range of resources available for learning and troubleshooting. It has a robust layout system that allows developers to create complex and responsive user interfaces that adapt to different screen sizes and resolutions. Additionally, PyQt provides a wide range of widgets and tools for building modern desktop applications, which will be touched upon in the next section [15]. All of these factors contributed to the decision to use PyQt for this project, as it provides a powerful and flexible framework for building a professional-quality desktop application with a modern user interface.

3.3 USER WORKFLOW

The concept behind the application's main purpose is not complicated - it takes in data, the user decides what exactly and how they want to display it, and the application generates a visualization based on those choices. The main workflow should therefore be very simple and intuitive to actually perform its intended purpose - to save time for engineers and technicians. The general workflow is depicted in Figure 3. 1.

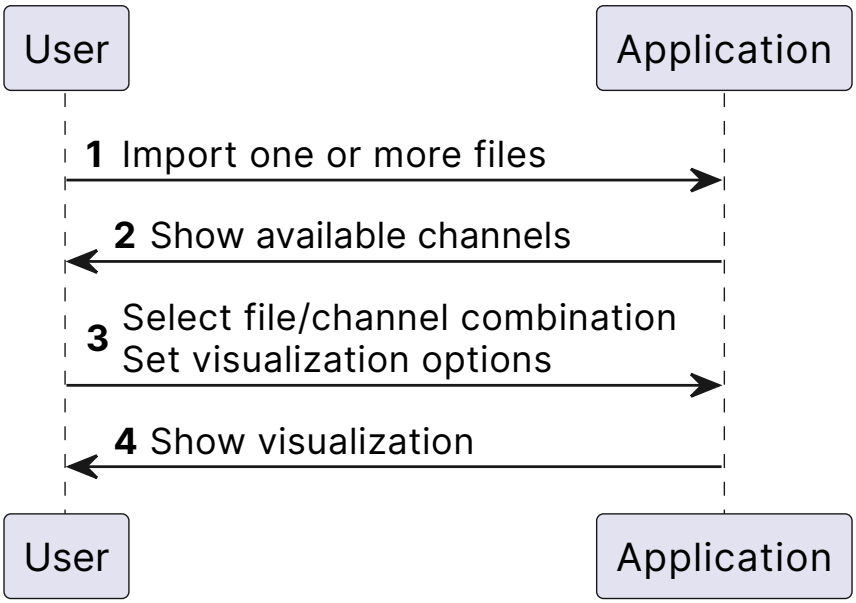


Figure 3. 1: General visualization workflow

4 BUILDING THE APPLICATION

4.1 DEVELOPMENT ENVIRONMENT

Development environment refers to the tools and software used by developers to write, test, and debug their code. A good development environment can significantly improve productivity and make the development process more efficient. For this project, the development environment consisted of a combination of an Integrated Development Environment (IDE) and various tools for version control, testing, and documentation.

The primary IDE of choice for this project was PyCharm, a popular Python IDE developed by JetBrains. PyCharm offers a wide range of features that enhance the development experience, including intelligent code completion, code navigation, and integrated debugging tools. A heavier IDE like PyCharm was chosen over lighter code editors such as Visual Studio Code or Sublime Text because of its robust support for Python and its powerful features that can help manage a larger codebase more effectively.

For the environment itself, the Nix package manager was used to create a reproducible development environment. Nix allows developers to define their development environment in a declarative way, specifying the exact versions of tools and libraries needed for the project. This ensures that all developers working on the project have a consistent environment, which can help avoid issues related to dependency conflicts or differences in tool versions while keeping everything isolated from the system environment [16]. The specific configuration used for this project is shown in Listing 2, which includes Python 3.11 and the necessary packages for development. Direnv was also used to automatically load the Nix environment when entering the project directory, further streamlining the development workflow [17].

```
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixpkgs-unstable";

  outputs =
    { nixpkgs, ... }:
    let
      system = "aarch64-darwin";
      pkgs = import nixpkgs { inherit system; };
    in
    {
      devShells.${system}.default = pkgs.mkShell {
        packages = with pkgs; [
          python311
          uv
        ];

        shellHook = ''
          if [ -f .venv/bin/activate ]; then
            source .venv/bin/activate
          else
            uv venv && uv sync && source .venv/bin/activate
          fi

          alias designer="pyqt6-tools designer &> /dev/null"
        '';
      };
    };
}
```

Listing 2: Nix development environment configuration

As the Nix configuration above shows, the package manager of choice for managing Python dependencies was uv, a modern Python package manager written in Rust that provides a fast and efficient way to manage Python packages and virtual environments [18]. It was chosen over more traditional tools like pip and virtualenv because of its speed, ease of use, and built-in support for dependency resolution and virtual environment management. The pyproject.toml file used by the project is shown in Listing 3, which specifies the project metadata and dependencies.

```
[project]
name = "graph-insights"
version = "1.10.0"
description = "Inspection tool for ODIS-exported Excel tabular data"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
  ...
]
```

Listing 3: The pyproject.toml file used by the project

Finally, Git was used for version control, more so for the sake of good practice and maintaining a history of changes rather than for collaboration, as the project was developed solely by the author. Git allows developers to track changes to their code, revert to previous versions if needed, and manage different branches of development. When working on a new feature or a large refactor, branching allows developers to isolate their changes from the main codebase until they are ready to be merged, which can help prevent conflicts, maintain a cleaner commit history, and allows the developer to work on multiple features or bug fixes simultaneously without interference [19]. For this project, a simple branching strategy was used, with a main branch for stable code and feature branches for new development. Regular commits were made to the feature branches, and once a feature was complete and tested, it was merged back into the main branch using the rebase strategy to maintain a clean commit history. The source code for the project was put up on a self-hosted GitLab instance, which provided a private repository for the project and allowed for easy access and management of the codebase.

4.2 PROJECT STRUCTURE

The project is structured as a single Python package named `graph_insights` containing all the source code for the application accompanied by the main entrypoint script `main.py`. Supporting files such as the UI design files, static assets, and documentation are organized into separate directories within the project.

Within the `graph_insights` package, the code is organized into several subpackages and modules based on functionality. For example, there are separate packages for data processing, visualization, and analysis, as well as a package for the user interface components (as shown in Listing 4).

```
-- graph_insights
  |--app/
  |--core/
  |--processing/
  |--qt/
  |--render/
  |--ui/
  |
  |---__init__.py
  |---__main__.py
```

Listing 4: Structure of the main package

This modular structure allows for better organization of the code and makes it easier to maintain and extend the application in the future. Each module contains related functions and classes that encapsulate specific functionality, following

the principles of separation of concerns and single responsibility. This organization also facilitates testing, as each module can be tested independently, and it allows for easier collaboration if other developers were to join the project in the future. The context of the various subpackages is outlined in Table 4. 1, which provides a brief description of the purpose of each directory within the `graph_insights` package.

Table 4. 1: Description of the `graph_insights` subpackages

Directory	Description
`app/`	Contains the main application logic and entry point.
`core/`	Contains core functionality and utilities used across the application.
`processing/`	Contains modules for data processing and analysis.
`qt/`	Contains modules related to the Qt framework and user interface components.
`render/`	Contains modules for rendering visualizations.
`ui/`	Contains the .ui files created with Qt Designer for the user interface design.

4.3 DATA INGESTION

When a user wants to add new data to the application, they can do so by selecting a file (or multiple files) from their file system using a file dialog. The application must then take these files and **asynchronously** load them in the background while keeping the user interface responsive. It must also inform the user about the progress of the loading process, so they are not left wondering if the application is frozen or if something went wrong. This is typically achieved by using a separate thread or process to handle the loading of data, while the main thread continues to run the user interface. The application can then use signals and slots (or other inter-thread communication mechanisms) to update the user interface with progress information and to notify the user when the loading process is complete.

I've chosen to go with a multithreaded pipeline approach for data ingestion. There are a number of distinct, ordered steps that need to be performed when loading a data file. Keeping all the logic for these steps in a single function would make it very difficult to read and maintin. It also makes it easier to unit test the individual steps of the loading process, as they can be tested in isolation from each other. The pipeline approach also allows for better error handling, as errors can be caught and handled at the appropriate stage of the loading process, rather than having to catch all errors in a single function. The complete list of stages in the data loading pipeline is shown in table Table 4. 2.

Table 4. 2: Stages of the data loading pipeline (in order)

Stage	Description
`read_from_excel`	Uses `pandas.read_excel()` to read in an Excel file to memory as a `DataFrame`
`extract_header_and_shift`	Parses the metadata string from the first row and discards it
`resolve_duplicate_columns`	Looks for duplicate column names in the `DataFrame`, removing all potential occurrences and logging apppropriately
`resolve_nameless_columns`	Looks for columns without a name in the `DataFrame`, removing all potential occurrences and logging apppropriately
`convert_dtypes`	Tries to convert columns to a numeric type, dropping any columns that cannot be converted
`reindex_to_timestamps`	Reindexes the `DataFrame` from sequential integers to timestamps, using the `#Time` column as the source
`drop_time_channel`	Drops the time channel from the `DataFrame`, as it is no longer needed after reindexing
`derive_channels`	Uses a predefined set of rules to derive new channels from existing ones
`normalize_column_names`	Normalizes column names to a consistent format (case and whitespace)

When an Excel file is ran through the pipeline, the `pandas` package is used to load it into a `DataFrame` object, which is a powerful data structure for handling tabular data and is the primary data structure used in the application. The Excel files we are working with have a specific format, where the first row contains metadata while the actual data starts from the second row. As per the example in Listing 5, the metadata contains the exact date and time when the record button was pressed, the internal identifier of the particular vehicle, and then an arbitrary number of space separated strings.

```
#2025-02-17 12:38:35 TMBNH9NY0SF000226 trasa MB-LIB-MB
```

Listing 5: Example of the metadata string in the first row of the Excel files

Following metadata extraction, the pipeline filters out possible duplicate and nameless columns, as these are not expected in the data and are likely to cause issues later on. The next stage is to try to convert all columns to a numeric type,

as this is the expected format for the data and it allows for more efficient storage and processing. Any columns that cannot be converted to a numeric type are dropped from the `DataFrame`, as they are not useful for our purposes. The logic behind this stage is depicted in Figure 4. 1.

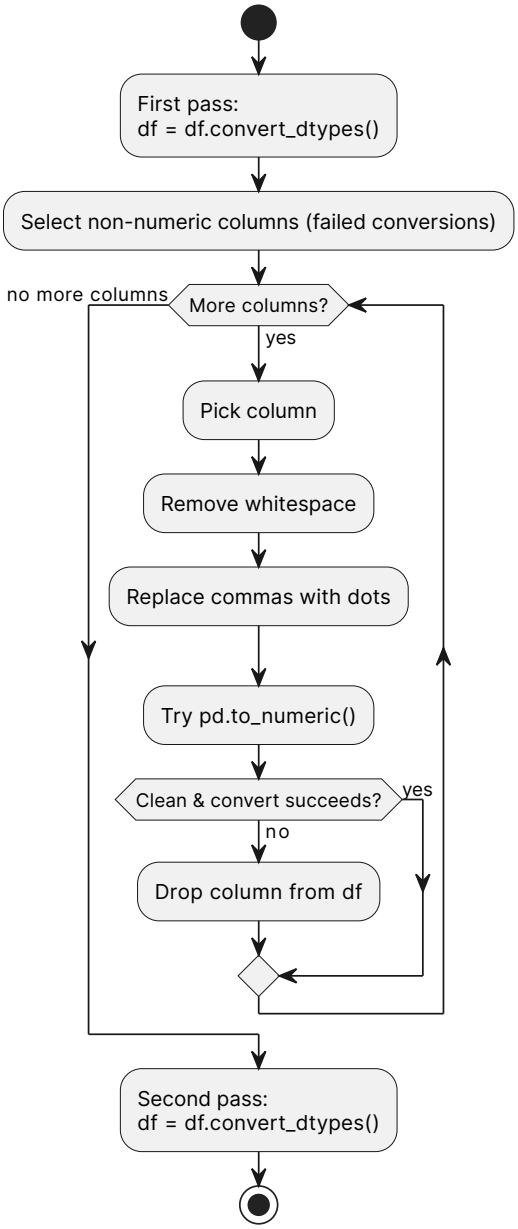


Figure 4. 1: Logic behing the `convert_dtypes` stage of the data loading pipeline

The next stage is to reindex the `DataFrame` from sequential integers to timestamps, using the `#Time` column as the source. This allows us to easily perform time-based operations on the data, such as resampling or rolling window calculations. After reindexing, the time channel is dropped from the `DataFrame`, as it is no longer needed. The next stage is to derive new channels from existing ones using a set of rules defined in `processing/derived_channels.py`. Most

commonly these include power calculation from voltage and current channels. Finally, the column names are stripped of whitespace and normalized to a consistent case (lowercase) to avoid issues with inconsistent naming conventions in the source data[2].

4.4 DESIGNING THE USER INTERFACE

The requirements for the user interface were to be simple, intuitive, and efficient for the target users, who are engineers and technicians at Škoda Auto. The application needed to provide a clear and organized way to visualize and analyze tabular data exported from ODIS, with a focus on usability and accessibility for users who may not have extensive programming experience.

There are two ways to design a user interface in Qt: using code or using a visual design tool, preferably the official Qt Designer. For very simple applications, setting up all the widgets and layouts in code can be straightforward and may even be quicker and easier than wiring up a design file. However, for more complex applications with multiple windows, dialogs, and a variety of widgets, using a visual design tool is a no-brainer, as it allows developers to quickly and easily create and modify the user interface without having to write a lot of code. Qt Designer is bundled with the PyQt6 package, so no need to install it separately. Recall Listing 2, where I setup an alias for the bundled Qt Designer, which allows me to simply type `designer` in the terminal to launch the Qt Designer application.

Each window or dialog in the application was designed as a separate .ui file using Qt Designer. A UI file is simply an XML file that describes the layout and properties of the widgets in a particular window or dialog. Because of its text-based nature, it can be easily version-controlled and merged using Git, which is a significant advantage over binary design files used by some other frameworks. The process of loading a UI file in Python is straightforward, as shown in Listing 6. The `.loadUi` function takes the path to the .ui file and the instance of the widget to load it into, and it automatically creates the necessary widgets and layouts as defined in the design file.

```
uic.loadUi(path.join(BASE_DIR, 'ui/analysis_overview.ui'), self)
```

Listing 6: Loading the UI file in Python

4.4.1 THE MAIN WINDOW

The main window of the application is also the most important one, as it serves as the central hub for all the functionality of the application and is the first thing

[2]Change in column names is a frequent occurrence when switching between different versions of data recording software.

that users see when they launch the application. It must provide the following key features:

- List of loaded datasets with the ability to add, remove, and switch between them
- A unionized list of all channels across all loaded datasets, with the ability to select which channels to visualize and analyze

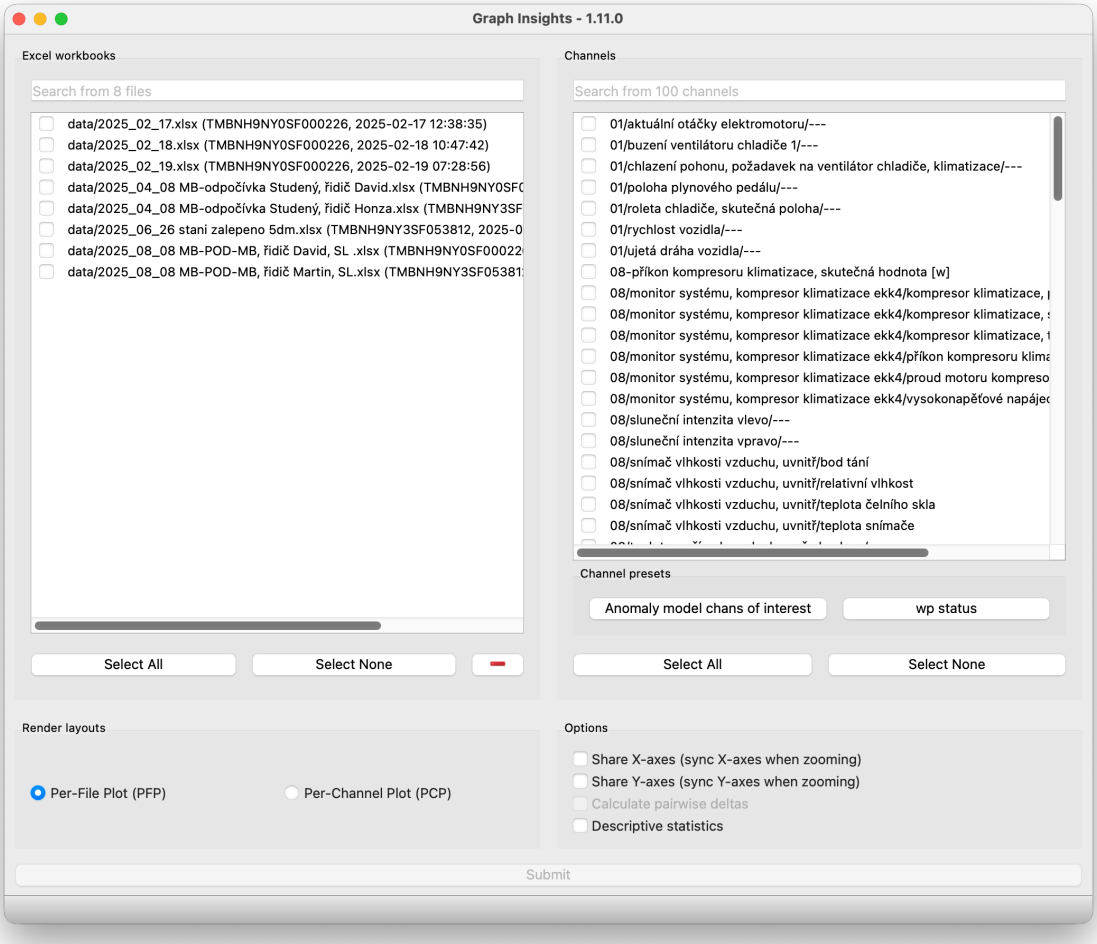


Figure 4. 2: The main window of the application, designed using Qt Designer

4.5 VISUALIZATION

Showing users a visual representation of their data is the core functionality of the application, so designing the visual part of the user interface was a crucial aspect of the project, both from the User Interface (UI) and User Experience (UX) perspective.

4.5.1 GRAPHING LIBRARY

Before diving into the specifics of the visualization system, a technology for the actual rendering of the visualizations had to be chosen. There are a number of options available for rendering visualizations in Python, each with its own advantages and disadvantages. A comparison of the most popular options available in the Python ecosystem is shown in Table 4. 3.

Table 4. 3: Comparison of visualization technologies in Python

Technology	Advantages	Disadvantages
Matplotlib	• Mature and widely used library with a large community and extensive documentation • Highly customizable and flexible, allowing for a wide range of visualization types and styles • Good performance for static visualizations	• Can be complex and difficult to learn for beginners • Not ideal for interactive visualizations or real-time data updates
Plotly	• Easy to create interactive visualizations with built-in support for zooming, panning, and tooltips • Can be used in both web applications and desktop applications using frameworks like Dash • Good performance for interactive visualizations	• Can be overkill for simple static visualizations • May have a steeper learning curve for advanced features
Bokeh	• Designed for creating interactive visualizations in web browsers • Can handle large datasets efficiently • Good performance for interactive visualizations	• Not ideal for static visualizations • May require additional setup for use in desktop applications

The table above is not an exhaustive comparison of all the visualization technologies available in Python, but it covers some of the most popular and widely used options. All of these are solid choices that provide all the necessary features for our use case, so the decision ultimately came down to personal preference and familiarity with the library. I drafted up possible UI designs for both Matplotlib

and Plotly and the results from the informal user testing sessions with potential users leaned towards **Matplotlib**, as it provided a more traditional and familiar visualization experience that was preferred by the target users. It also helped that I've had a few years of experience working with Matplotlib, namely for university assignments and personal projects.

With the rendering technology chosen, the next step was to design a flexible and extensible system for creating and managing visualizations within the application. The main goal was to create a system that would allow users to easily create and customize their visualizations, while also providing a solid foundation for future development and expansion of the application's visualization capabilities.

4.5.2 SYSTEM ARCHITECTURE

The rendering system is quite complex, as it needs to handle a wide range of use cases and requirements. For that reason, I chose to break it up into multiple parts (along with the diagrams) for the purposes of this thesis. A diagram of the system as a whole is shown in Attachment 6, which provides a high-level overview of the different components and their interactions.

At the heart of the system is the `Renderer` class. It works by instantiating a `Renderer` object with a certain specification, and then calling the `render()` method to generate the visualization. The `Renderer` class is designed to be flexible and extensible, allowing for different rendering backends to be used (e.g., Matplotlib, Plotly, etc.) and for different layout strategies to be implemented (e.g., grid layout, stacked layout, etc.). The architecture of the core and data components is depicted in Figure 4. 3.

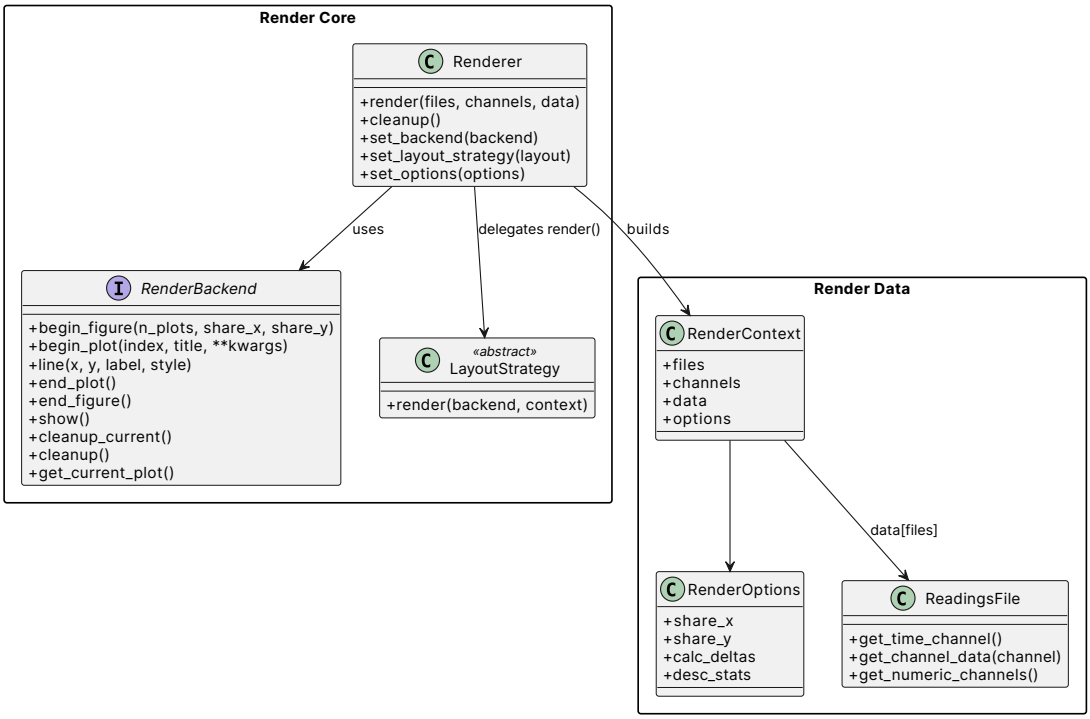


Figure 4. 3: Architecture of the rendering system, showing the core components and their interactions

A `RenderBackend` is the unified API for a specific rendering technology that provides a consistent interface for the rest of the application to interact with, regardless of the underlying rendering technology being used. This allows for greater flexibility and modularity in the design of the application, as different rendering technologies can be easily swapped out or added without requiring significant changes to the rest of the codebase. It has a state-machine-like design with the functions being a little biased towards Matplotlib's way of doing things, as it is the primary rendering technology used in the application, but it is designed to be flexible enough to accommodate other rendering technologies in the future if needed.

`LayoutStrategy` is responsible for determining how the visualizations are arranged and displayed within the application. There are basically two main layout strategies implemented in the application: **Per-File Plot (PFP)** and **Per-Channel Plot (PCP)**. The PFP strategy creates a separate plot for each selected dataset containing all selected channels, while the PCP strategy creates a separate plot for each selected channel containing all selected datasets. Both strategies have their own advantages and disadvantages, and the choice between them ultimately comes down to user preference and the specific use case. The architecture of the backend and layout components is depicted in Figure 4. 4.

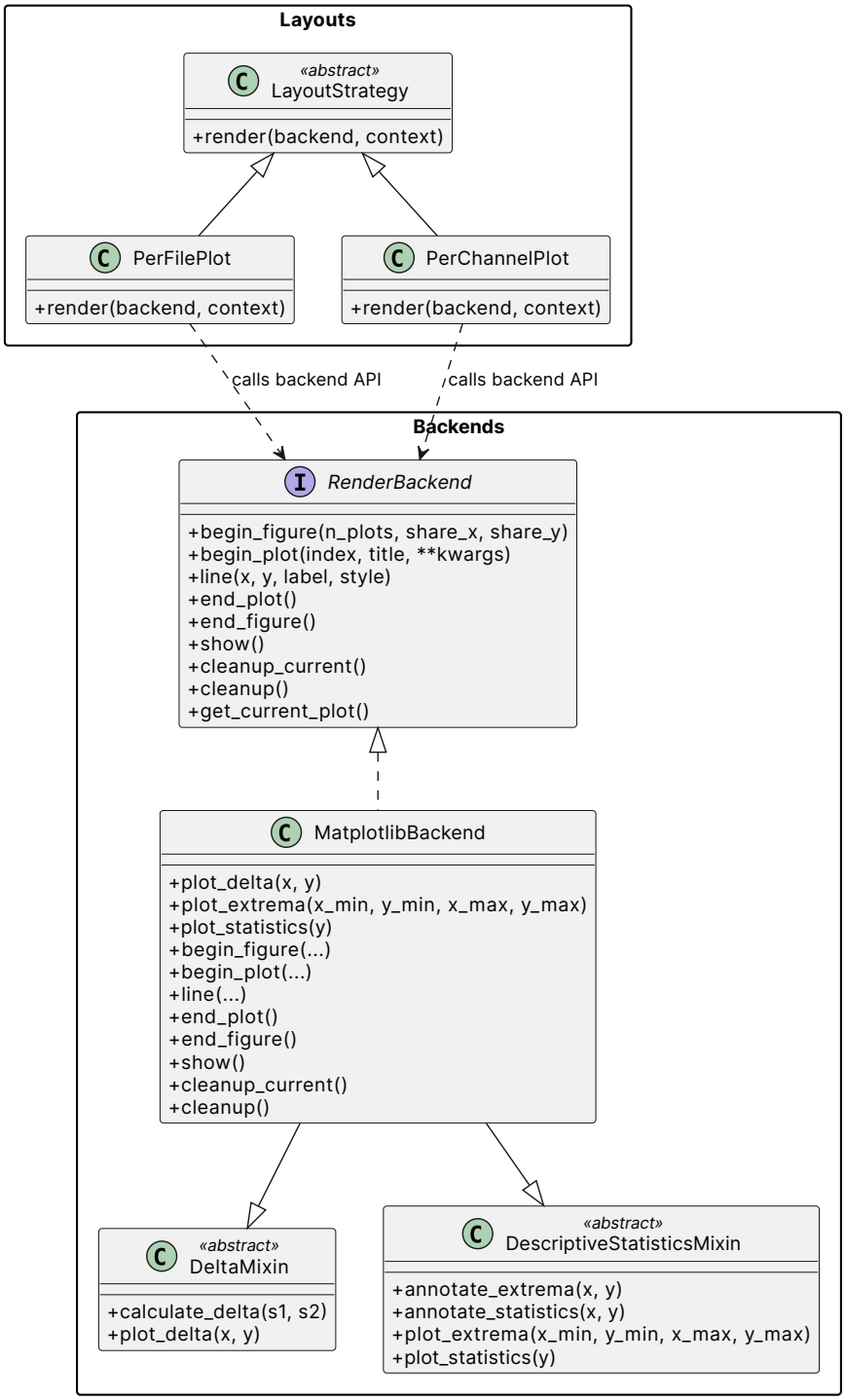


Figure 4. 4: Architecture of the rendering system, showing the backend components and their interactions

4.5.3 MIXINS

A *mixin* is by definition a class that provides methods to other classes through inheritance, but is not intended to be instantiated on its own [20]. In the rendering system, mixins are used to extend the functionality of a `RenderBackend` by

exposing additional methods that are specific to the mixin's functionality and also having the backend implement the necessary logic to support those methods - mainly rendering.

During the development, user feedback was collected and contributed to key features being added to the application, such as the ability to display the difference between two curves on a plot - a delta curve. This feature was implemented as a mixin (`DeltaMixin`) which exposes a `calculate_delta()` method that takes in two curves, performs the necessary calculations to compute the delta curve and returns it. It also defines an abstract method `plot_delta()` that must be implemented by any backend that uses the mixin, which takes care of rendering the delta curve on the plot. This design allows for a clear separation of concerns, as the mixin is responsible for the logic of calculating the delta curve, while the backend is responsible for rendering it. The architecture of the `DeltaMixin` is depicted in Figure 4. 5.

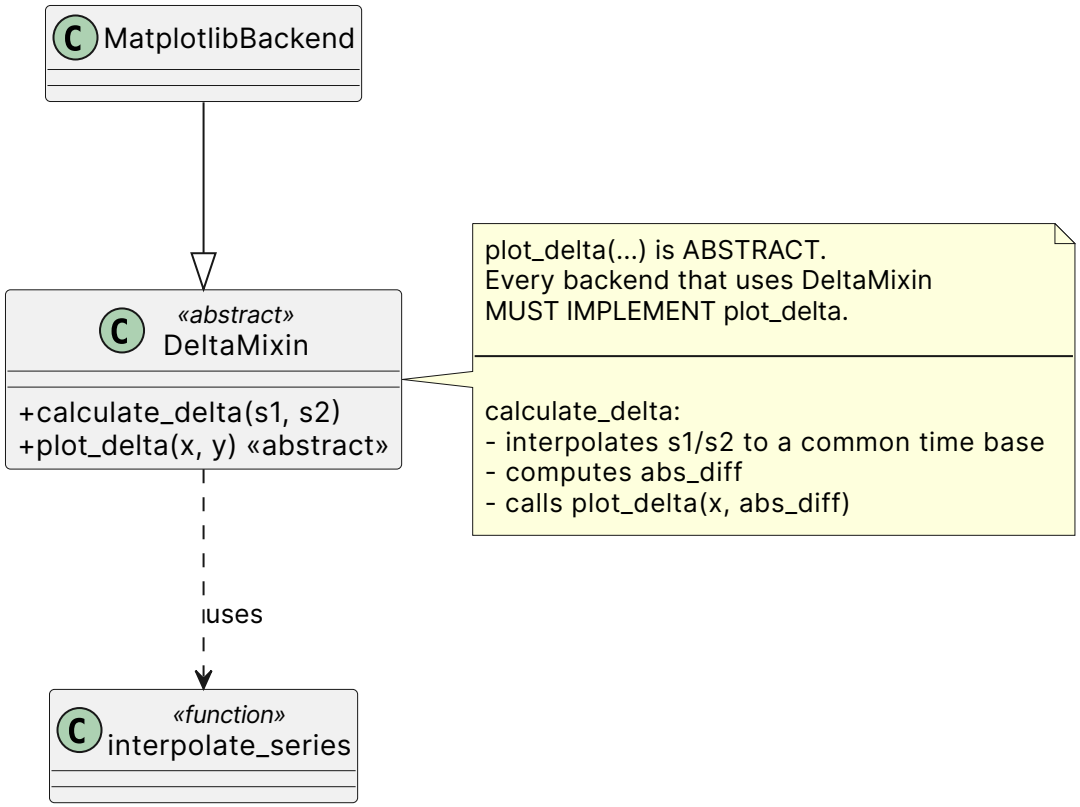


Figure 4. 5: `DeltaMixin` architecture

In practice, when a user selects the option to display the delta curve on a plot, the application does a few preliminary checks, namely whether the current context[3] and backend support it and if so, it calls the `calculate_delta()` method with

[3]A user selection involves zero or more files and channels. Depending on the layout, plots may contain a different number of curves. A delta curve is only calculated between exactly two curves.

the appropriate curves. The backend then performs the necessary calculations and calls the `plot_delta()` method to render the delta curve on the plot. The particular implementation of the `plot_delta()` method also adds a baseline to the plot at y=0 and dedicates the right y-axis to the delta curve, which allows for better visualization and interpretation of the delta curve in relation to the original curves. A concrete example of a plot with the delta curve rendered is shown in Figure 4. 6.

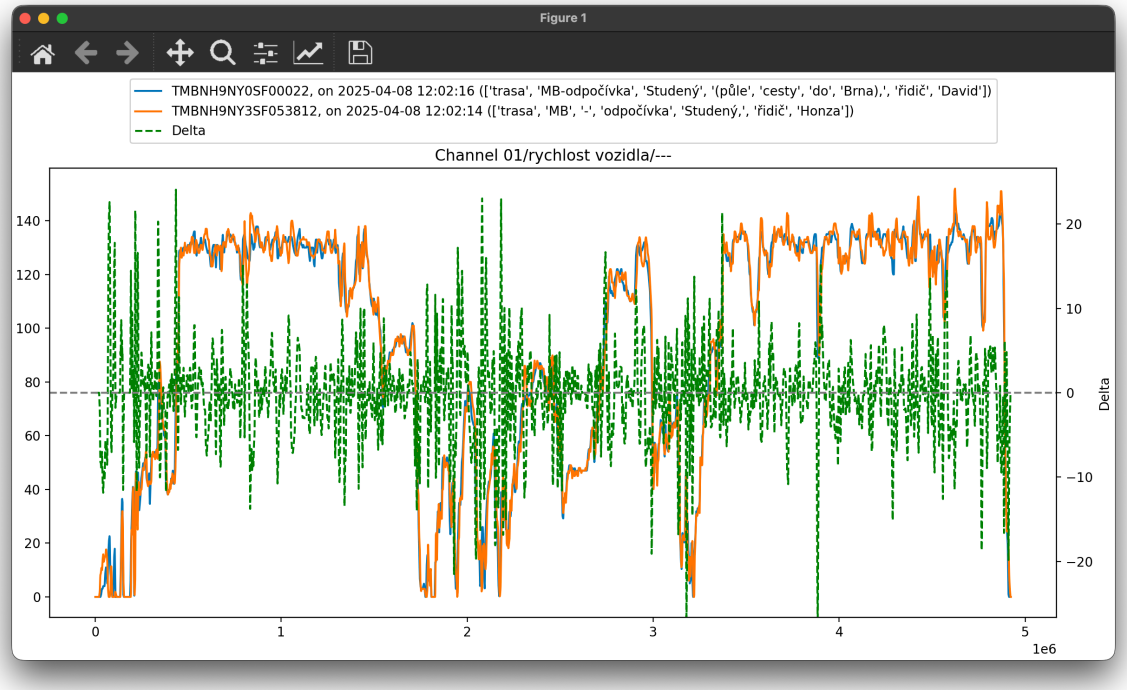


Figure 4. 6: Delta curve for two vehicles' speed channel

A need for in-plot statistical analysis features was also identified during user testing sessions, which led to the implementation of a `DescriptiveStatisticsMixin` that provides methods for calculating and displaying descriptive statistics such as mean, median, standard deviation, etc. for the selected curves on a plot. Similar to the `DeltaMixin`, it defines abstract methods (`plot_extrema()` and `plot_statistics()`) that must be implemented by any backend that uses the mixin, which take care of rendering. The architecture of the `DescriptiveStatisticsMixin` is depicted in Figure 4. 7.

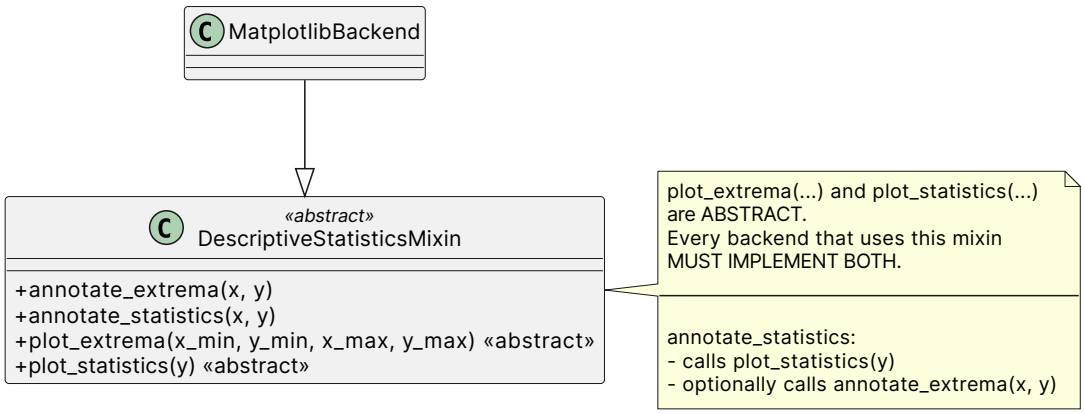


Figure 4. 7: `DescriptiveStatisticsMixin` architecture

As an example of how the `DescriptiveStatisticsMixin` works in practice, when a user selects the option to display descriptive statistics on a plot, the application first checks if the current backend supports it and if so, it calls the appropriate methods to calculate the statistics for the selected curves. The backend then performs the necessary calculations and depending on the number of curves[4] in the plot, it may call the `plot_extrema()` method to render markers for the extrema points of each curve, and/or the `plot_statistics()` method to render a table with the calculated statistics on the plot. A concrete example of a plot with descriptive statistics rendered is shown in Figure 4. 8.

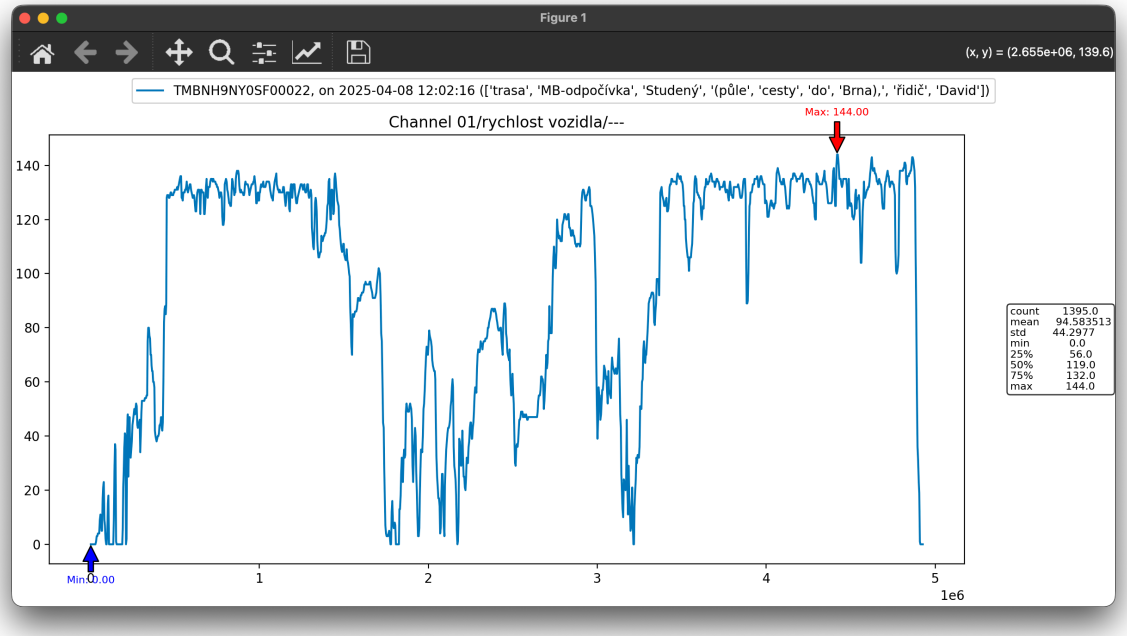


Figure 4. 8: Descriptive statistics for the speed channel of a single vehicle

[4] If a plot contains a single curve, global extrema are annotated and descriptive statistics are displayed. In case of multiple curves, only the extrema are annotated for each curve.

4.5.4 PLUGINS

After the core rendering system was implemented and in use by the users, the need for additional features and customizations arose. To accommodate these needs without cluttering the core rendering system with too many features that may not always be relevant, a plugin architecture was implemented. It allows us to inject additional functionality into the rendering system at different stages of the rendering process without modifying the core codebase. This design promotes modularity and separation of concerns, as plugins can be developed and maintained independently from the core rendering system, while still being able to interact with it in a well-defined way.

The concept as well as the implementation itself is straightforward. A plugin is simply a callable object such as a function or a lambda, that takes in an instance of a `RenderBackend` and performs some operations on it. Different stages of the rendering process (e.g., figure creation, plot creation, etc.) get a `pre` and `post` plugin hooks assigned to them by annotating the functions themselves with the `@lifecycle` decorator[5], as shown in Listing 7.

```
@lifecycle("begin_figure")
def begin_figure(self, ...):
  ...

@lifecycle("begin_plot")
def begin_plot(self, ...):
  ...
```

Listing 7: Plugin hooks for the rendering stages

A plugin can then be registered to any of these hooks, and it will be called at the appropriate time during the rendering process. The architecture of the plugin system is depicted in Figure 4. 9.

[5]The 'pre' hook is called before the stage is executed, and the 'post' hook is called after the stage is executed automatically, the decorator only requires the hook name.

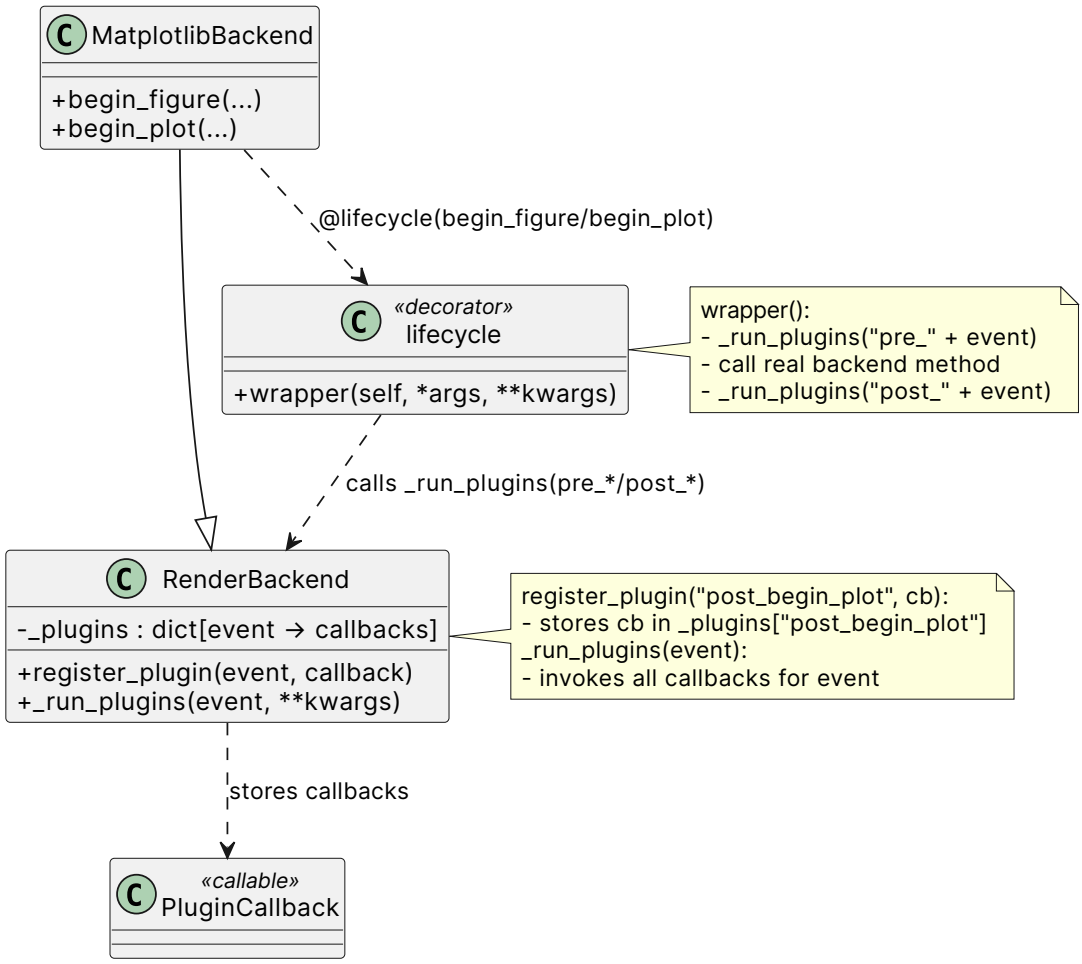


Figure 4. 9: Backend plugin architecture

To give a more tangible example, consider a discrete flag channel (as opposed to a normal continuous channel) that indicates the status of a certain system in the vehicle, in this case the state of the heat pump. First, the channel itself is derived in the data processing stage (recall Section 4.3) from a combination of multiple valves' position channels. Then, a plugin is registered to the `post_begin_plot` hook, as shown in Listing 8, that checks if the heat pump channel is present in the plot's context and if so, it translates the discrete values of the channel to human-readable status labels and renders them as the y-axis ticks on the right y-axis of the plot.

```
renderer.backend.register_plugin('post_begin_plot', wp_status_ytics)
```

Listing 8: Plugin registration (assuming `renderer` is an instance of `Renderer`)

4.6 ANALYSIS

The application with just the visualization features implemented already provided a lot of value to the users, as it allowed them to easily visualize and explore their data in a way that was not possible before. However, the ability to automatically extract insights and information from the data is what really takes the workflow to the next level. The goal is the following: as a user adds and removes files from the application during its lifetime, the application should be able to automatically analyze the data in the background and extract insights from it, which are then presented to the user in a clear and organized way. This takes the necessary time consuming process of inital screening of the data off the users' shoulders and allows them to focus on the actual analysis and interpretation of the data, which is where their expertise lies.

I have split the process of analysis into two categories: **individual** and **comparative**[6]. This means that files are analyzed both individually and in comparison to each other, which targets different use cases and allows for a more comprehensive analysis of the data.

Because the threaded pipeline pattern described and used in Section 4.3 worked so well for the data loading process, I decided to use a similar approach for the analysis features as well. Whenever a new file is added to the application, it goes through the processing pipeline, and once it is loaded and processed, the `on_processing_pipeline_batch_finished()` callback is triggered, which submits the file into the `IndividualAnalysisPipeline` for individual analysis. As a result, `submit_file_pairs_for_analysis()` is also called, which generates the file combinations and submits them into the `FilePairAnalysisPipeline` for comparative analysis. Both pipelines work in a separate thread and have their own set of stages that are executed in order, similar to the data loading pipeline. The results of the analysis are then stored in a cache for quick access and are also emitted as signals to update the user interface with the new insights.

4.6.1 CHANNEL DATA CORRELATION

In some scenarios, it may be useful to check whether certain channels are roughly (or exactly) the same across multiple files, maybe from a test drive conducted on multiple vehicles driven at the same time. Hypothetically, a technician may want to confirm that both vehicles' radiator shutters (or a blind-like alternative like the AAB [21] that are more common nowadays in Škoda vehicles) behaved the same during the test drive, which could be an indication of a potential issue with the shutters if they did not. This is where correlation analysis comes in handy. By calculating the correlation coefficient between the channel

[6]The term comparative in this context denotes a strictly pairwise framework, wherein analysis is conducted exclusively between two distinct files at a time.

`01/roleta chladiče, skutečná poloha/---`[7] across the corresponding files, the application can provide a quick and easy way for the user to verify whether the shutters behaved correctly or not.

There are a number of different correlation coefficients that can be used to measure the correlation between two channels, such as Pearson's correlation coefficient, Spearman's rank correlation coefficient, and Kendall's tau coefficient. Each of these coefficients has its own advantages and disadvantages, and the choice of which one to use depends on the specific characteristics of the data and the analysis being performed. For this application, I chose the **Pearson's correlation** coefficient as it is a widely used measure of linear correlation between two variables and is appropriate for the type of data being analyzed.

As far as the implementation goes, this feature falls under the category of comparative analysis, as it involves comparing channels across multiple files. Whenever a new file is added to the application, a set of distinct file combinations is generated using the `itertools.combinations()` function from the Python standard library. If the set of currently loaded files is denoted by F, then the number of combinations grows according to the following formula:

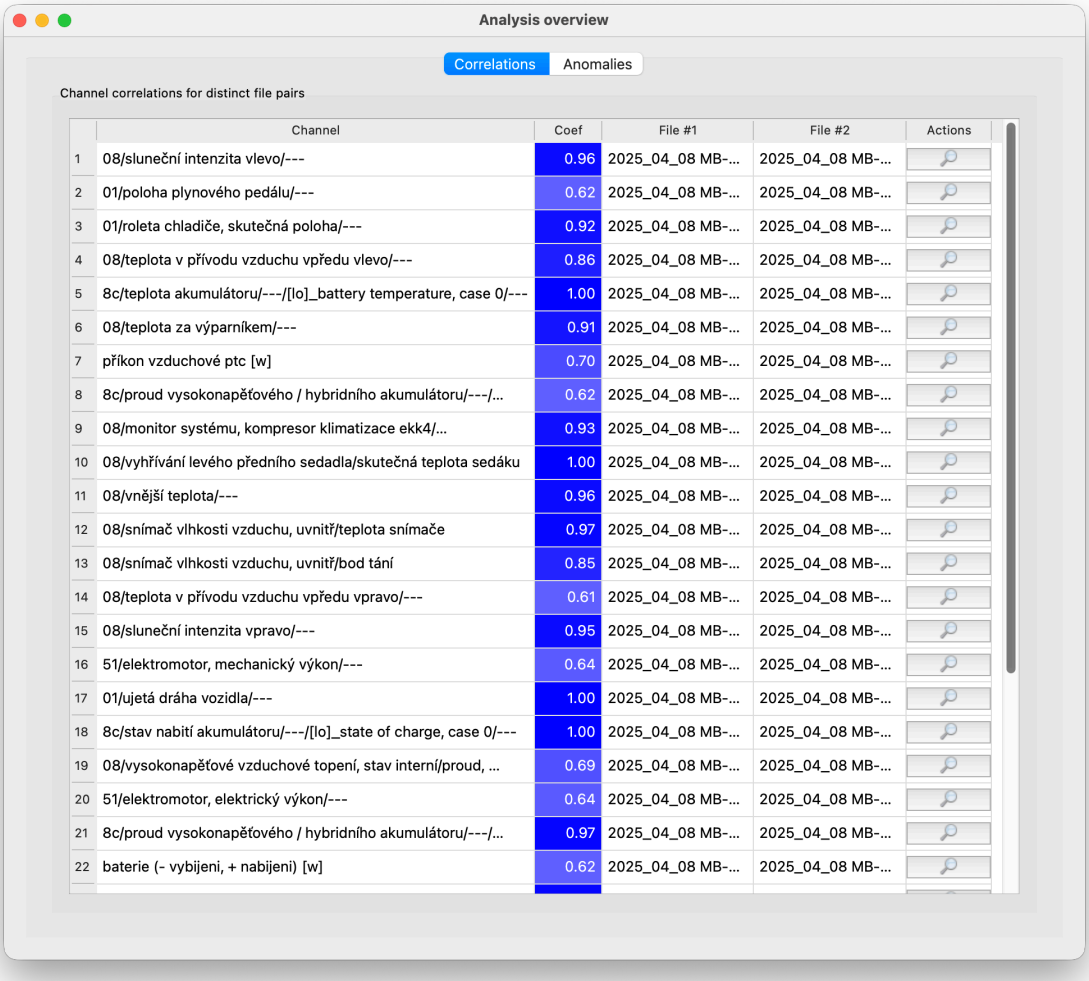
$$\binom{|F|}{2} = \frac{|F|!}{2! * (|F| - 2)!}$$

This might seem like a lot of combinations, but in practice the number of files loaded at the same time is usually quite small (2-5), so it is not a problem.

The set of files is then submited into the `FilePairAnalysisPipeline`. The pipeline holds a cache of previously calculated correlation coefficients for each pair of files, so if a combination has already been analyzed before, the cached result is returned immediately without having to recalculate it. If the combination is new, the pipeline calculates the correlation coefficient for each pair of channels between the two files and stores the results in the cache for future reference.

For visualization purposes, the results are presented to the user as a table containing the list of correlated channels across multiple files, showing the correlation coefficient (color coded from red to blue, with red indicating strong negative correlation and blue indicating strong positive correlation) and a button to quickly view the two channels in question on a plot beside each other for a more detailed comparison, as shown in Figure 4. 10. The table columns are sortable, giving the user the ability to quickly find the most correlated channels across their files, which can be a useful starting point for further analysis and investigation. Additional forms of visualization for the correlation results are discussed in Section 6.

[7] Internal data is often in a combination of Czech, English and German



	Channel	Coef	File #1	File #2	Actions
1	08/sluneční intenzita vlevo/---	0.96	2025_04_08 MB-...	2025_04_08 MB-...	
2	01/poloha plynového pedálu/---	0.62	2025_04_08 MB-...	2025_04_08 MB-...	
3	01/roleta chladiče, skutečná poloha/---	0.92	2025_04_08 MB-...	2025_04_08 MB-...	
4	08/teplota v přívodu vzduchu vpředu vlevo/---	0.86	2025_04_08 MB-...	2025_04_08 MB-...	
5	8c/teplota akumulátoru/---/[lo]_battery temperature, case 0/---	1.00	2025_04_08 MB-...	2025_04_08 MB-...	
6	08/teplota za výparníkem/---	0.91	2025_04_08 MB-...	2025_04_08 MB-...	
7	příkon vzduchové ptc [w]	0.70	2025_04_08 MB-...	2025_04_08 MB-...	
8	8c/proud vysokonapěťového / hybridního akumulátoru/---/...	0.62	2025_04_08 MB-...	2025_04_08 MB-...	
9	08/monitor systému, kompresor klimatizace ekk4/...	0.93	2025_04_08 MB-...	2025_04_08 MB-...	
10	08/vyhřívání levého předního sedadla/skutečná teplota sedáku	1.00	2025_04_08 MB-...	2025_04_08 MB-...	
11	08/vnější teplota/---	0.96	2025_04_08 MB-...	2025_04_08 MB-...	
12	08/snímač vlhkosti vzduchu, uvnitř/teplota snímače	0.97	2025_04_08 MB-...	2025_04_08 MB-...	
13	08/snímač vlhkosti vzduchu, uvnitř/bod tání	0.85	2025_04_08 MB-...	2025_04_08 MB-...	
14	08/teplota v přívodu vzduchu vpředu vpravo/---	0.61	2025_04_08 MB-...	2025_04_08 MB-...	
15	08/sluneční intenzita vpravo/---	0.95	2025_04_08 MB-...	2025_04_08 MB-...	
16	51/elektromotor, mechanický výkon/---	0.64	2025_04_08 MB-...	2025_04_08 MB-...	
17	01/ujetá dráha vozidla/---	1.00	2025_04_08 MB-...	2025_04_08 MB-...	
18	8c/stav nabití akumulátoru/---/[lo]_state of charge, case 0/---	1.00	2025_04_08 MB-...	2025_04_08 MB-...	
19	08/vysokonapěťové vzduchové topení, stav interní/proud, ...	0.69	2025_04_08 MB-...	2025_04_08 MB-...	
20	51/elektromotor, elektrický výkon/---	0.64	2025_04_08 MB-...	2025_04_08 MB-...	
21	8c/proud vysokonapěťového / hybridního akumulátoru/---/...	0.97	2025_04_08 MB-...	2025_04_08 MB-...	
22	baterie (- vybijeni, + nabijeni) [w]	0.62	2025_04_08 MB-...	2025_04_08 MB-...	

Figure 4. 10: List of correlated channels across multiple files

4.6.2 STATISTICAL METHODS FOR ANOMALY DETECTION

Comparative analysis can be very useful for extracting insights from the data, but it cannot determine whether a particular file contains any **anomalies** or not. Be it invalid readings, unexpected behavior of a certain system in the vehicle, or just a generally weird file that doesn't really fit in with the rest of the data, it is important to be able to automatically flag these files for the user so they can take a closer look at them or even exclude them from the analysis if they are deemed to be of low quality. This is where the need for anomaly detection stems from.

Anomaly detection is a technique used to identify unusual patterns or behaviors in data that do not conform to expected norms. In the context of this application, anomaly detection is used to identify parts of channel data that deviate

significantly from the expected behavior, which could indicate potential issues or areas of interest for further analysis. This section focuses on purely statistical methods for anomaly detection, while the next one explores machine learning applications.

There are numerous tools and techniques statistics provides for inconsistency detection, such as outlier detection methods (e.g., Z-score, IQR), time series analysis techniques (e.g., ARIMA, STL decomposition), and change point detection algorithms (e.g., PELT, Binary Segmentation). Each of these methods has its own advantages and disadvantages, and the choice of which one to use depends on the specific characteristics of the data and the type of anomalies being targeted. For this application, I implemented a simple standard deviation-based outlier detection method, which identifies data points that are a certain number of standard deviations, away from the mean as anomalies (the exact implementation is shown in Listing 9).This method is computationally efficient and works well for normally distributed data, which our data roughly is after the processing stage. It tells us which data points are anomalous for a particular channel in a particular file. Formally, the method can be described as follows:

Let $X = \{x_1, x_2, ..., x_n\}$ be the set of observed values for a given channel. We compute the sample mean and standard deviation as:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \quad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

An anomaly detection threshold is then defined as:

$$T = \mu + k\sigma$$

where k is a chosen constant (in this case $k = 3$).

A data point x_i is classified as an anomaly if:

$$x_i > T$$

```
for channel in file.get_numeric_channels():
  channel_data = file.get_channel_data(channel)
  mean = channel_data.mean()
  std_dev = channel_data.std()
  threshold = mean + 3 * std_dev
  anomaly_points = np.where(channel_data > threshold)[0].tolist()
```

Listing 9: Standard deviation thresholding logic used in the application

The results are again presented to the user in a tabular fashion, showing the list of channels in a file that contain anomalies (and the amount) along with a button to quickly view the channel on a plot with the anomalous points highlighted for a more detailed analysis. Because this method falls under the category of individual analysis, the results are shown on a per-file basis by utilizing Qt's tab

widget, which allows for a clear and organized presentation of the results while also providing an easy way for users to navigate between different files and their corresponding analysis results, as shown in Figure 4. 11.

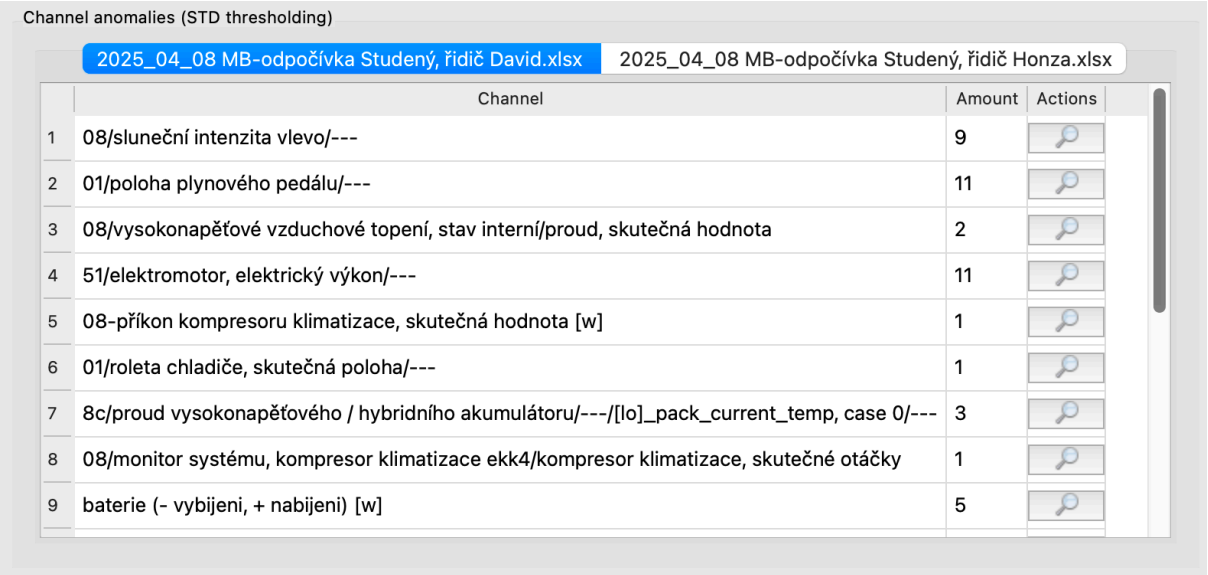


Figure 4. 11: Visualization of the found anomalies using the standard deviation thresholding method

As mentioned, a button is provided for each channel with anomalies that allows users to quickly view the channel on a plot with the anomalous points highlighted. The results from the analysis pipeline are in the form of a list of indices corresponding to the anomalous data points in the channel. To make this data more interpretable and actionable for the users, the application constructs intervals from data points directly adjacent to each other, as these are likely to be part of the same anomaly, and highlights these intervals on the plot using a shaded area. A concrete example of such a plot is shown in Figure 4. 12, where the anomalous intervals are highlighted in red.

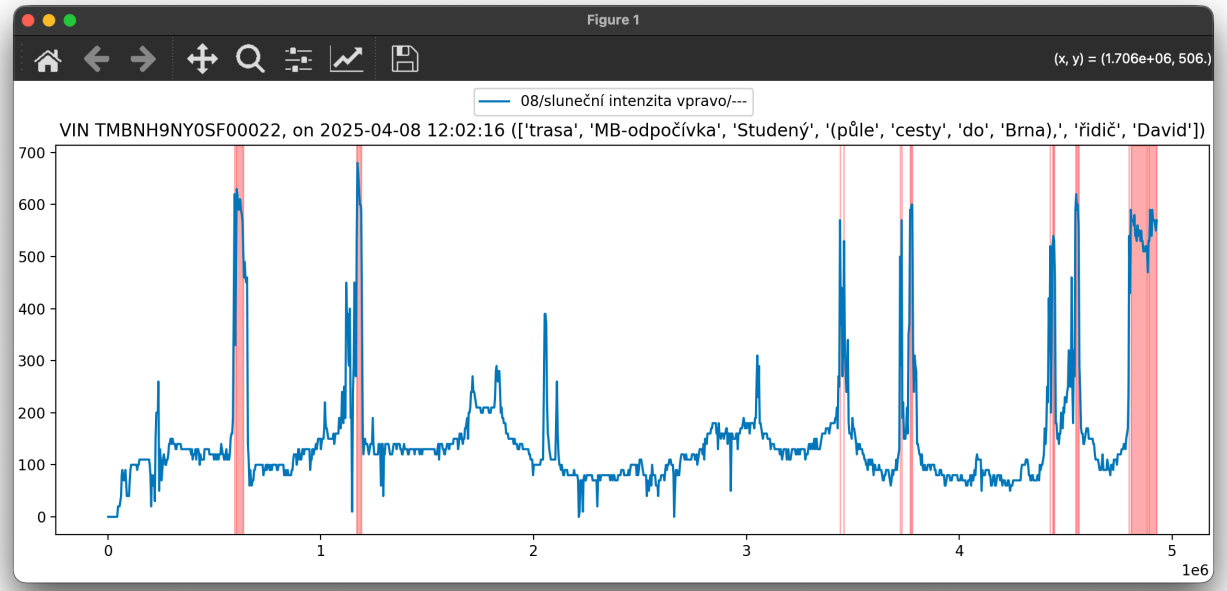


Figure 4. 12: Highlighted anomalous intervals on a plot using STD thresholding

4.6.3 MACHINE LEARNING FOR ANOMALY DETECTION

The statistical methods described in the previous section are a good starting point for anomaly detection, and can reliably identify certain types of anomalies in the data. However, their entire data context is just the files loaded into the application, so any inconsistencies that may be found are only relative to the other data in the application, and not necessarily in an absolute sense. For example, it cannot reveal illogical values in a channel that are not necessarily outliers in the context of the other loaded files. This is where Machine Learning (ML) can lend a helping hand. We keep pretty much all the data from previous test drives on a shared drive, so we have a large amount of historical data that can be used to train ML models to identify anomalies in the data based on patterns and relationships that may not be immediately apparent through statistical methods alone. By leveraging ML techniques, the application can provide a more robust and comprehensive anomaly detection system that can help users identify potential issues and areas of interest in their data more effectively.

ML algorithms can be broadly categorized into supervised and unsupervised learning methods. Supervised learning methods require labeled data for training, which means that the anomalies in the historical data would need to be identified and labeled by experts, which can be a time-consuming and labor-intensive process. Unsupervised learning methods, on the other hand, do not require labeled data and can identify anomalies based on patterns and relationships in the data itself. Given that we want to dump a bunch of historical data into the

model and have it train itself without the need for manual labeling, **unsupervised** learning methods are the way to go for our use case. A couple of popular unsupervised ML algorithms for anomaly detection are compared in Table 4. 4.

Table 4. 4: Comparison of a few unsupervised machine learning algorithms for anomaly detection

Algorithm	Core Idea	Pros	Cons
Isolation Forest	Random partitioning isolates anomalies with fewer splits	• Excellent scalability • Handles high-dimensional data well • Minimal assumptions • Robust in practice	• Less intuitive than simple distance-based methods
Local Outlier Factor (LOF)	Detects anomalies via local density deviation	• Good for local structure	• Struggles in high dimensions • Parameter sensitive • Slower than tree-based methods
K-Means (distance-based)	Distance from centroid determines anomaly score	• Simple and fast	• Assumes simple cluster shapes • Less robust than isolation-based methods
PCA-based Detection	Projection and reconstruction error	• Efficient baseline method	• Limited to linear relationships • Less robust than isolation-based approaches

I ultimately decided to go with the Isolation Forest algorithm, as it is a powerful and widely used method for anomaly detection that is particularly effective for high-dimensional data, which is the case with our datasets. It works by randomly partitioning the data and isolating anomalies with fewer splits, which allows it to effectively identify anomalies even in complex datasets. The implementation of

the Isolation Forest algorithm is provided by the `scikit-learn` library, which makes it ridiculously easy to use and integrate into the application.

I implemented the training and validation of the model in a separare script that is run independently from the main application, as the training process can be quite time-consuming and resource-intensive, and also as not to convolute the main application codebase with ML-specific logic. The script simply takes in a location on disk where the individual files are stored and a combination of channels, which defines the model's dimensionality, and trains the model on the data from these channels across all the files.

I used `scikit`'s `Pipeline` class to create a pipeline that includes data preprocessing steps (e.g., scaling, dimensionality reduction) and the Isolation Forest algorithm itself. The trained model is then saved to disk using `joblib` which users just point the application at in the settings and the application takes care of loading it and using it for anomaly detection (the whole pipeline is extracted from the `.joblib`, so the implementation on the application side stays very lean). This also allows for the possibility of training multiple models on different combinations of channels or different subsets of the data (e.g., winter vs. summer data), and easily switching between them in the application settings without having to modify the codebase.

This allows the application to point out anomalies in the sense of "Is this combination of channel values something out of the ordinary given what we have measured in the past?" which can be a very powerful tool for identifying potential issues and areas of interest in the data that may not be immediately apparent through statistical methods alone.

For example, a model trained on the joint behavior of coolant temperature, vehicle speed, and radiator fan activity can flag measurements in which the cooling system behaves unlike the historical norm, even if none of the individual channels would appear suspicious when inspected in isolation. Such a result can draw attention to unusual thermal behavior during a specific drive and provide a useful starting point for a more detailed engineering investigation.

The results are again presented to the user in a tabular fashion (illustrated in Figure 4. 13), showing the list of files that contain anomalies based on the model's predictions along with a button to quickly view the relevant channels on a plot with the anomalous points highlighted the same way as with the statistical method for a more detailed analysis, as shown in Figure 4. 14.

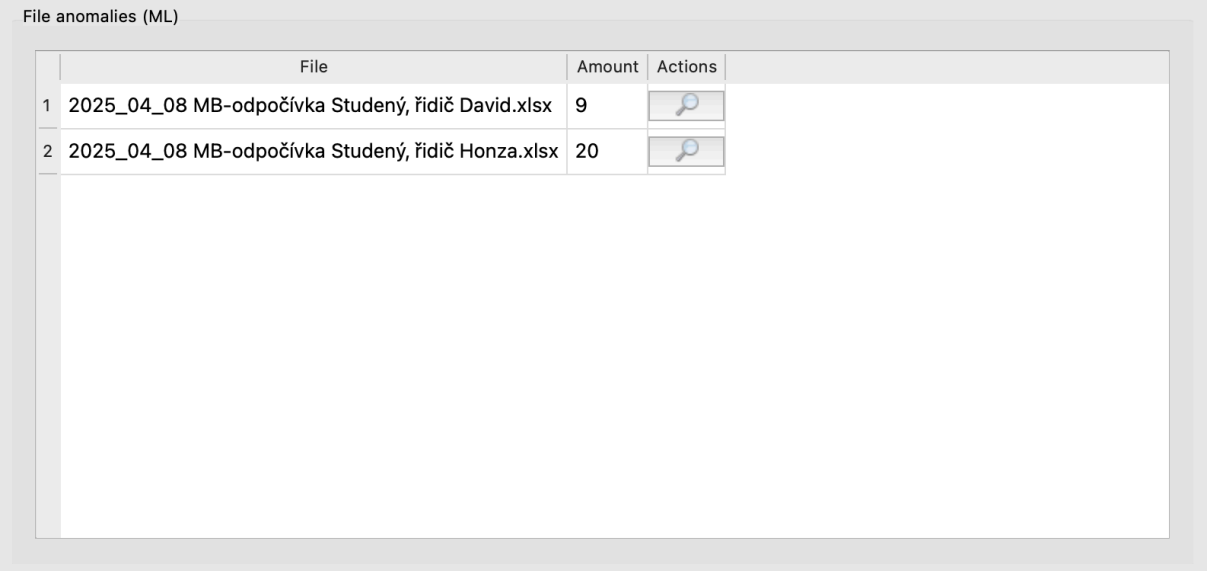


Figure 4. 13: Visualization of anomalies detected using a machine learning method

Figure 4. 14: Highlighted anomalous points on a plot using ML (dual-channel model)

5 DEPLOYING TO PRODUCTION

A crucial aspect of software development is ensuring that your application can be easily and reliably deployed to your users. In this chapter, I will cover the various aspects of deployment, including user documentation, testing, and packaging.

5.1 USER DOCUMENTATION

For any software application, especially a highly technical one, user documentation is essential. It helps users understand how to use the application effectively on their own. In this section, I will focus on the user documentation pertaining to the usage of the application, rather than the technical details of how it works that would be more relevant for developers.

There were various options for the format of the documentation I considered, such as markdown files, a static website, or even a PDF manual. I ultimately decided to go with a static website. Over the years, I have used MkDocs for generating documentation websites, and I found it to be a great tool for this purpose. It allows for easy writing in markdown, and it generates a clean and professional-looking website with many Quality of Life (QoL) features built-in. Most of the time, I have used MkDocs in conjunction with the Material for MkDocs theme, which provides a modern and responsive design. The maintainers behind the Material for MkDocs have been working on a project called **Zensical**, which is a documentation generator that aims to provide an even better experience for both writers and readers of documentation. The configuration is very similar (almost backwards compatible) to MkDocs, reducing the learning curve for people like me [22]. I have therefore decided to give Zensical a try for this project.

When it comes to hosting the documentation, there are pretty much two options: hosting it locally or on the cloud (just someone else's computer). The latter is generally more convenient for users, as they can access the documentation from anywhere without needing to download it. However, hosting on the cloud brings significant baggage in terms of cost and maintenance. Even though it is simple static content that could just be hosted on S3 or GitHub Pages, I wanted to avoid the hassle of setting up and maintaining a hosting solution. I instead opted to host the documentation locally, or more specifically, to include the documentation as part of the application itself. This way, users can access the documentation directly from the application without needing an internet connection or worrying about hosting costs.

The way this works is that the documentation website is generated as a static site bundle, which is then included in the application's resources. When the application is started, a local web server is spun up to serve these files on a random port assigned to us by the Operating System (OS). Even better, no additional dependencies are required to serve the documentation, as Python's standard library includes the `http.server` module, which makes all of this possible.

The documentation includes a quick start guide, which provides users with the essential information they need to get started with the application (illustrated by Figure 5. 1). It also includes a more detailed user guide that covers all the features of the application in depth, focusing on the two main use cases: visualization and data analysis. The website has both an english and a czech version, and users can easily switch between the two languages using a language selector in the top right corner of the page.

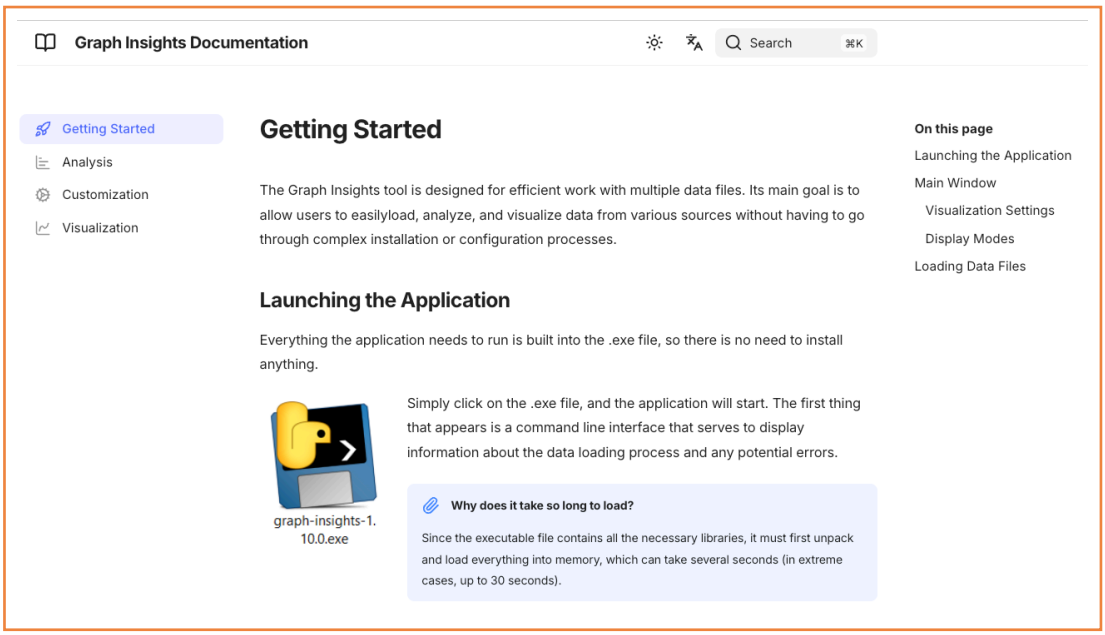


Figure 5. 1: Home page of the documentation website

5.2 TESTING

Virtually every piece of software contains bugs, and no software is perfect. However, by implementing a robust testing strategy, we can significantly reduce the number of bugs that make it into production and ensure that our application behaves mostly as expected across changes and updates. Testing can take many forms, including unit tests, integration tests, end-to-end tests, and user testing.

In this project, I focused primarily on unit tests, which are tests that verify the behavior of individual components or functions in isolation. Unit tests are typically fast to run and can be easily automated. All unit tests are under the

`tests` directory in regular python modules as functions prefixed with `test_`. I used the pytest framework for writing and running the tests, which provides a simple and powerful way to write tests in Python [23]. To measure the test coverage, I used the `coverage` package, which allows me to see which lines of code are covered by tests and which are not [24]. This helps me identify areas of the code that may need more testing. I was not aiming for abnormally high test coverage, because most of the results the application provides are visual and not easily testable with automated tests. Instead, I focused on testing the core logic and functionality of the application, while relying on user testing to validate the visual aspects and overall user experience.

As of version 1.11.0, the test coverage is around 70%, which I consider to be a good balance between ensuring code quality and not getting bogged down in writing tests for every single line of code.

As mentioned, the user testing was also an important part of the testing strategy. I released versions of the application to users and on a weekly basis, I sat down with them to gather feedback and observe how they were using the application. This provided valuable insights into how users were interacting with the application, what features they found useful, and what areas needed improvement and what features were missing.

5.3 Versioning and Release Management

As described in Section 4.1, the project started with version control in mind, using Git from the very beginning. This allowed me to keep track of all changes made to the codebase and to easily revert back if thing went wrong. As far as commit messages go, I sticked to the *Conventional Commits* specification, which provides a standardized format for commit messages [25]. This makes it easier to understand the history of the project and to generate changelogs automatically.

As development progressed, I released the software to users for testing and feedback (more on release artifacts in Section 5.4). I versioned these releases using *Semantic Versioning*, which uses a three-part version number (major.minor.patch) to indicate the level of changes made in each release [26]. During the early stages of ML analysis, I even employed a release candidate (RC) versioning scheme, which allowed me to indicate that a release was still in the testing phase and not yet ready for production use. This helped manage user expectations and encouraged them to provide feedback on the release. At the time of writing, the project is at version 1.11.0, and still in active development. I have been using annotated Git tags to mark the specific commits that correspond to each release, which makes it easy to track the history of releases and to roll back to a previous version if necessary (see Listing 10).

```
git tag -a v1.11.0 -m "Release version 1.11.0"
```

Listing 10: Tagging a release commit with Git (annotated tag)

5.4 PACKAGING

Every time a release is made, the application needs to be packaged for distribution. The requirements for packaging the application were quite specific. I wanted to create a single executable file that users could easily run as is. Installing any kind of software on the company provided machines is a bureaucratic nightmare, and I wanted to avoid asking users to go through that process just to use the application. That meant that I needed to bundle all the dependencies and resources into a single file that could be executed without any additional setup. Furthermore, I wanted the application to be cross-platform. Despite that 99% of the users are on Windows, I still wanted to support potential users on other operating systems, such as macOS and Linux.

For packaging Python applications into standalone Windows executables (.exe files), several tools are commonly used, including PyInstaller and Nuitka. During the initial release process, both options were evaluated; however, due to technical difficulties encountered with Nuitka, PyInstaller was ultimately selected as the preferred solution. This decision was further influenced by time constraints, as a functional proof of concept was already available using PyInstaller and an initial release needed to be delivered within a limited timeframe.

PyInstaller uses the current platform as the target platform for the generated executable. This means that to create a Windows executable, I have to run PyInstaller on a Windows machine, which sadly can't be easily automated in a CI/CD pipeline. However, using PowerShell scripts, I was able to streamline the packaging process into more or less a single command (the PyInstaller configuration used by those scripts is shown in Listing 11). The final executable generated to the `dist` directory is then manually moved to a shared drive that users have access to, where a versioned folder is created for each release.

```
def install():
  PyInstaller.__main__.run([
      path_to_main,
      '--add-data', 'ui/*:ui',
      '--add-data', 'pyproject.toml:.',
      '--add-data', 'docs/cs/site:docs/cs/site',
      '--add-data', 'docs/en/site:docs/en/site',
      '--onefile',
      '--clean',
      '--distpath', os.path.join(BASE_DIR, 'dist', RELEASE_VERSION),
      '--collect-submodule', 'sklearn',
      '--name', f'graph-insights-{RELEASE_VERSION}',
  ])
```

Listing 11: PyInstaller configuration for packaging the application into a single executable file

One notable drawback of this approach is the resulting size of the executable. Because the packaging process bundles all dependencies, along with the Python interpreter itself, the final executable is relatively large (approximately 150 MB). This increase in size is an inherent consequence of consolidating all required resources into a single distributable file. Additionally, the application exhibits a longer cold start time (upwards of 30 seconds), as the bundled components must first be unpacked and the Python runtime initialized before execution of the application code can begin.

Despite my negative perspective on these issues, the collected feedback from users has been overwhelmingly positive, with no significant complaints regarding the size of the executable or the startup time. This suggests that, for the target user base, the convenience of having a single executable file outweighs the drawbacks associated with its size and startup performance.

6 POSSIBILITIES FOR IMPROVEMENT

I'd like to dedicate this chapter to my thoughts on how the current state of the project could be improved or how I would do things differently if I were to start from scratch. This is not meant to be a critique of the current implementation, but rather a reflection on potential areas for enhancement and future development.

6.1 DATA PROCESSING

In the never ending process of optimization and trying to make things faster, there are a few areas that I would focus on. One being able to recognize and skip already processed files. This would be particularly useful when dealing with large datasets, as it would save time and computational resources by avoiding redundant processing. Implementing a caching mechanism that stores the results of previously processed files could be an effective way to achieve this. By hashing the contents of the files and storing the processing pipeline results, we could quickly determine if a file has already been processed and retrieve the cached results instead of reprocessing it. User feedback on this particular feature was pretty much a green light, so I will definitely prioritize it in the next iterations of the project.

Another possible improvement in the core processing pipeline would be to leverage distributed computing frameworks like Apache Spark. The company issued laptops are not exactly powerhouses (at least the ones not dedicated for heavy workloads such as CAD) and as the datasets grow larger, processing times can become a bottleneck. By integrating distributed computing capabilities, we could offload some of the processing tasks to a cluster of machines, allowing for faster data processing and analysis. It also helps that Škoda Auto already has a well-established infrastructure for big data processing, so integrating Spark into the project would be a natural fit. This would not only improve performance but also enable us to handle larger datasets more efficiently, ultimately leading to more insightful analyses and better decision-making based on the data.

6.2 VISUALIZATION

Even though the rendering framework is a fairly recent redesign, there are still some aspects that could be improved. For instance, the figure management

system is currently quite basic, and for a simple reason - we only handle single figures at the moment. This limits our ability to create more complex visualizations that could span across multiple figures or subplots. In the future, I would consider implementing a more robust figure management system that allows for greater flexibility and scalability in our visualizations.

As mentioned in Section 4.6.1, the visualization portion of the correlation-based analysis (and also the other types) is currently quite basic. We simply create a list of the most correlated features and display them in a table. While this provides some insight, it lacks depth and interactivity. Personally, I would like to see a more dynamic and interactive visualization that allows users to explore the correlations in more detail. This could include features such as basic correlation matrices (as shown in Figure 6. 1), interactive scatter plots, or even heatmaps that visually represent the strength of correlations between features.

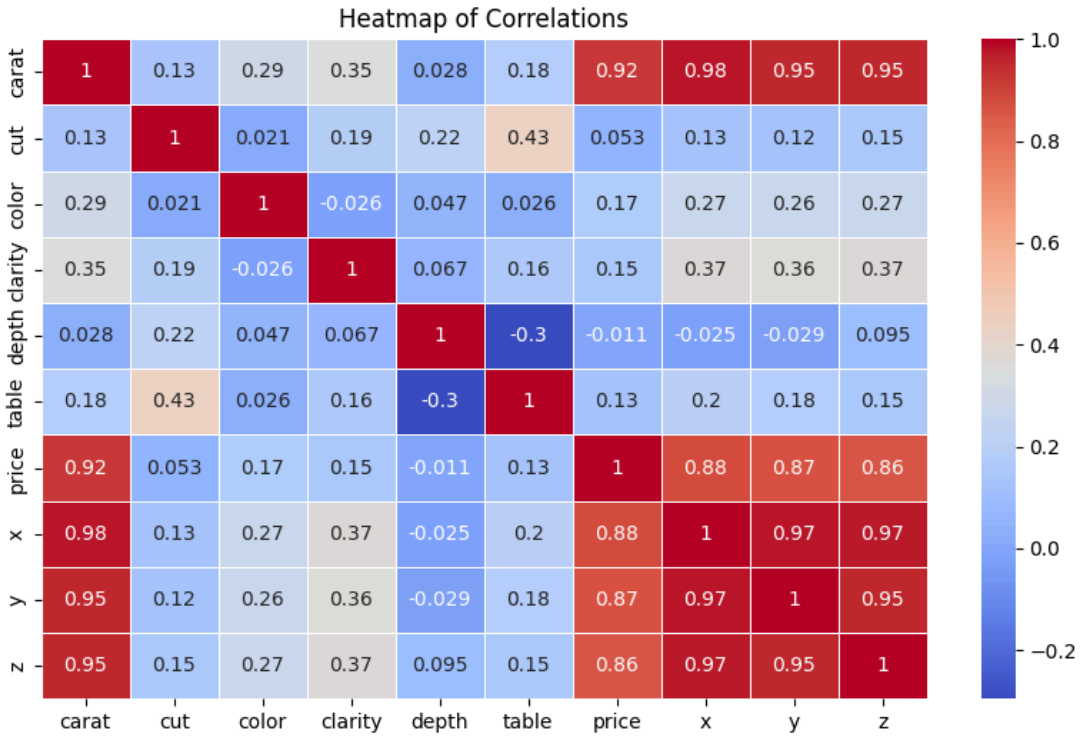


Figure 6. 1: Example of a correlation matrix using Matplotlib.
Source: https://codesignal.com/learn/courses/feature-engineering-and-correlation-analysis-in-pandas/lessons/heatmap-of-correlation-matrix

Such enhancements would not only make the analysis more engaging but also provide deeper insights into the relationships between features. These proposed changes also have the additional benefit of being relatively straightforward to implement, making them a practical next step in the development of the project.

6.3 MISCELLANEOUS

Lastly I have some thoughts on the technologies and tools used in the project. Python is a great choice for data processing and analysis. The development speed and the rich ecosystem of libraries made prototyping and iterating on the project much faster than it would have been with a lower-level language. However, were I to start from scratch, I might consider using a more performant language like C++. The user interface is currently built using PyQt, which is just a set of Python bindings for the Qt framework written in C++. By using C++ directly, we could potentially achieve better performance while using the same underlying framework for the UI. This would allow us to maintain the same level of functionality while improving the overall responsiveness and efficiency of the application.

Additionally, using C++ could open up opportunities for further optimizations in the data processing pipeline, as we would have more control over memory management and low-level operations. However, it's important to weigh these potential benefits against the increased development time and complexity that comes with using a lower-level language, especially considering the current success and functionality of the project as it stands.

7 CONCLUSION

This thesis addressed a practical problem arising in the post-processing of telemetry data from automotive test drives. In the examined workflow at Skoda Auto, engineers and technicians already have access to large amounts of measured data, yet extracting useful information from it remains unnecessarily slow and repetitive. As shown in the chapters **Problem analysis** and **Building the application**, the main bottleneck is not the acquisition of the data itself, but the later processing of tabular exports, repeated comparison of measurements, and the lack of a lightweight tool tailored to everyday engineering analysis.

Based on these findings, I evaluated existing categories of software used in automotive and telemetry-oriented environments, including professional engineering platforms, OBD-II diagnostic tools, and general telemetry analysis solutions. The survey confirmed that powerful products already exist, but also showed that they do not fully match the needs of the target users in this case. Many are designed for ECU development, proprietary measurement ecosystems, or large-scale enterprise deployments. Others are easier to access, but remain focused on diagnostics rather than efficient comparative analysis of exported test-drive data. In the given context, a dedicated application represented the most suitable solution.

The design part of the thesis therefore focused on the architecture of a multi-platform desktop application capable of supporting this workflow. I chose Python because of its rapid development cycle and strong data-processing ecosystem, while PyQt provided a mature framework for the desktop user interface, as discussed in **Designing the software architecture**. An important architectural contribution is the separation of concerns between data loading and transformation, application logic, rendering, and user interface components, which is reflected in the implementation of the data-ingestion pipeline in Section 4.3 and in the rendering subsystem described in the **Visualization** section.

The practical result of the thesis is a working software application that goes beyond a proof of concept. The implemented solution supports loading and preprocessing ODIS-exported data, normalization of channels, handling of multiple datasets, and visualization in layouts suitable for different comparison tasks. Beyond standard plotting, the application also includes analytical functions that directly reflect the identified needs of users: descriptive statistics and delta-curve visualization in the **Visualization** section, correlation-based comparison in Section 4.6.1, statistical anomaly detection based on standard deviation thresholding, and an unsupervised machine-learning extension in the **Analysis** section.

The resulting application is therefore not only a viewer of data, but also a tool for finding suspicious behavior and useful relationships within measurements.

An important part of the work was also verifying that the application can be used in practice. For that reason, the thesis covered documentation, testing, versioning, and packaging in addition to implementation itself, as described in **Deploying to production** and especially in Section 5.4. The project was supported by user documentation, a reproducible development environment, automated tests for the key parts of the codebase, and a release process resulting in a standalone executable suitable for distribution in a restrictive corporate environment. Regular feedback from users influenced both the interface and the analytical features, and confirmed that practical usability is just as important as technical sophistication. At the same time, the thesis also identified several natural directions for further development, which are summarized in Section 6.

BIBLIOGRAPHY

[1] J1962_201607: Diagnostic Connector. Online. 12 July 2016. [Accessed 16 March 2026]. Available from: https://saemobilus.sae.org/standards/j1962_201607-diagnostic-connector

[2] ISO 15031-5:2015 Road vehicles — Communication between vehicle and external equipment for emissions-related diagnostics — Part 5: Emissions-related diagnostic services. Online. August 2015. [Accessed 16 March 2026]. Available from: https://www.iso.org/standard/66368.html

[3] Offboard Diagnostic Information System Engineering (ODIS Engineering). Online. [Accessed 16 March 2026]. Available from: https://www.dne-elektronik.de/en

[4] INCA software products: Tools for measurement, ECU calibration, and diagnostics. Online. [Accessed 16 March 2026]. Available from: https://www.etas.com/ww/en/products-services/data-acquisition-processing-tools/software-products/inca-software-products/

[5] ASAM MDF (Measurement Data Format). Online. [Accessed 16 March 2026]. Available from: https://www.asam.net/standards/detail/mdf/

[6] CANoe (Vector Informatik GmbH) — ASAM member directory. Online. [Accessed 16 March 2026]. Available from: https://www.asam.net/members/detail/vector-informatik/

[7] CANape (Vector Informatik GmbH) — ASAM product directory. Online. [Accessed 16 March 2026]. Available from: https://www.asam.net/members/product-directory/detail/canape/

[8] VCDS-Lite: Windows-based Diagnostic Software for VW / Audi / Seat / Skoda. Online. [Accessed 16 March 2026]. Available from: https://www.register.ross-tech.com/vcds-lite/index.php

[9] OBDAssistant: AI-powered OBD2 vehicle intelligence platform. Online. [Accessed 16 March 2026]. Available from: https://www.obd2assistant.com/

[10] AutoPi Cloud and automotive data loggers. Online. [Accessed 16 March 2026]. Available from: https://www.autopi.io/

[11] Welcome to asammdf's documentation! — asammdf 8.7.2 documentation. Online. [Accessed 16 March 2026]. Available from: https://asammdf.readthedocs.io/en/latest/

[12] Introduction | Electron. Online. [Accessed 23 March 2026]. Available from: https://electronjs.org/docs/latest

[13] MCKINNEY, Wes. *Python for Data Analysis*. "O'Reilly Media, Inc.", 2013. ISBN 978-1-4493-1979-3.

[14] Matplotlib — Visualization with Python. Online. [Accessed 27 November 2025]. Available from: https://matplotlib.org/

[15] Qt (software). Online. 2026. [Accessed 24 March 2026]. Available from: https://en.wikipedia.org/w/index.php?title=Qt_(software)&oldid=1343148945Page Version ID: 1343148945

[16] How Nix Works | Nix & NixOS. Online. [Accessed 25 March 2026]. Available from: https://nixos.org/guides/how-nix-works/

[17] direnv – unclutter your .profile. Online. [Accessed 25 March 2026]. Available from: https://direnv.net/

[18] uv. Online. [Accessed 25 March 2026]. Available from: https://docs.astral.sh/uv/

[19] Git. Online. [Accessed 25 March 2026]. Available from: https://git-scm.com/about

[20] Mixin. Online. 2026. [Accessed 6 April 2026]. Available from: https://en.wikipedia.org/w/index.php?title=Mixin&oldid=1345632593Page Version ID: 1345632593

[21] Active Air Blinds | Röchling EN. Online. [Accessed 19 April 2026]. Available from: https://www.roechling.com/automotive/products-solutions/aerodynamics/active-air-blinds

[22] Zensical. Online. [Accessed 21 April 2026]. Available from: https://zensical.org/

[23] pytest documentation. Online. [Accessed 27 April 2026]. Available from: https://docs.pytest.org/en/stable/

[24] Coverage.py — Coverage.py 7.13.5 documentation. Online. [Accessed 27 April 2026]. Available from: https://coverage.readthedocs.io/en/7.13.5/

[25] Conventional Commits. Online. [Accessed 22 April 2026]. Available from: https://www.conventionalcommits.org/en/v1.0.0/

[26] PRESTON-WERNER, Tom. Semantic Versioning 2.0.0. Online. [Accessed 22 April 2026]. Available from: https://semver.org/

ATTACHMENTS

1. GitLab repository for the project: https://gitlab.com/pycrs-epa/graph-insights
2. Instrumented engine bay *(attached bellow)*
3. Instrumented cockpit *(attached bellow)*
4. Installed ETAS hardware *(attached bellow)*
5. Top-down view of ETAS hardware *(attached bellow)*
6. Rendering system architecture *(attached bellow)*
7. Miscellaneous *(attached bellow)*

ATTACHMENT 2

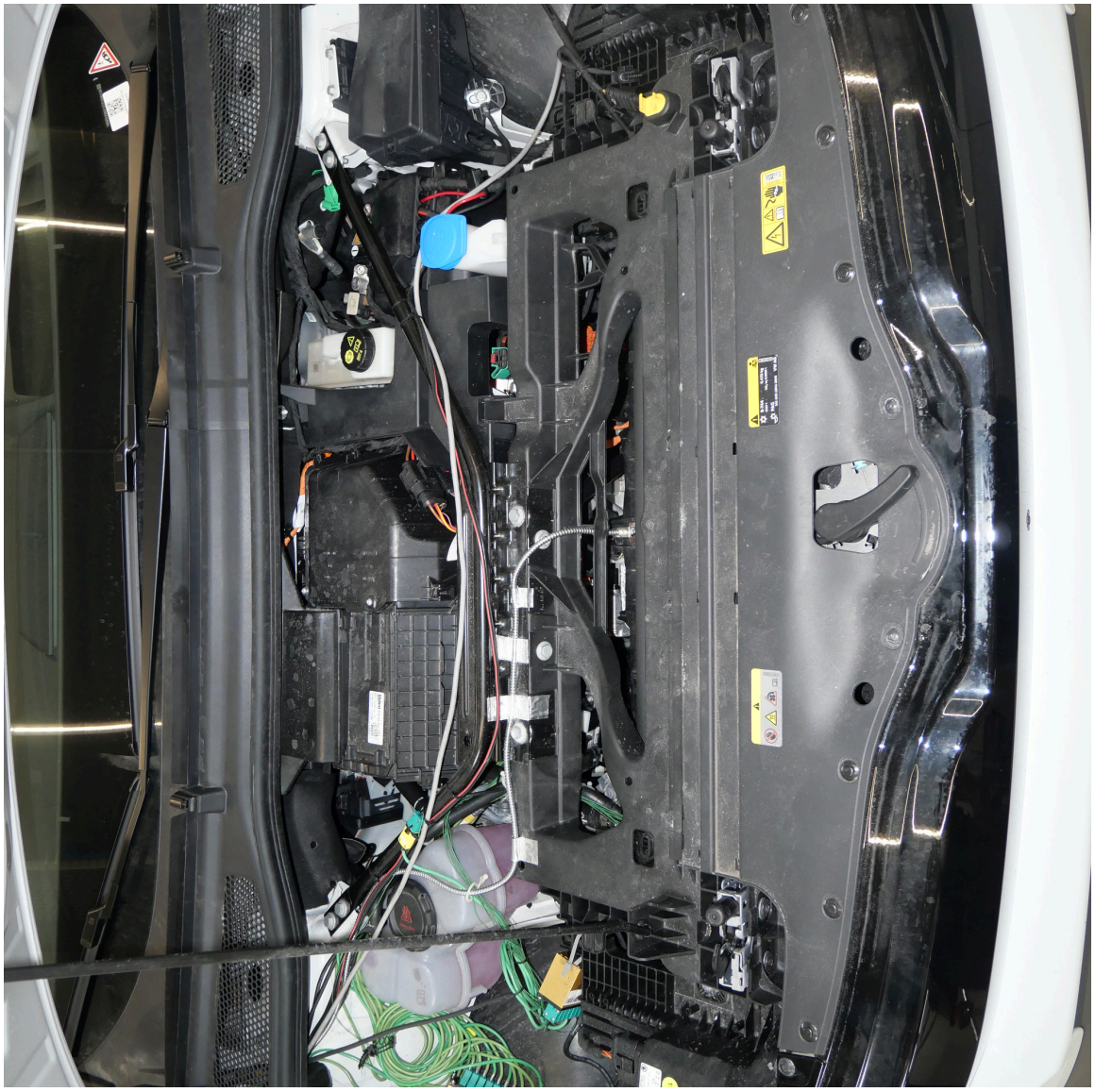


Figure 7. 1: The engine bay of a test vehicle instrumented with additional sensors for data acquisition.

ATTACHMENT 3

Figure 7. 2: State of the vehicle's cockpit during test drives.

ATTACHMENT 4

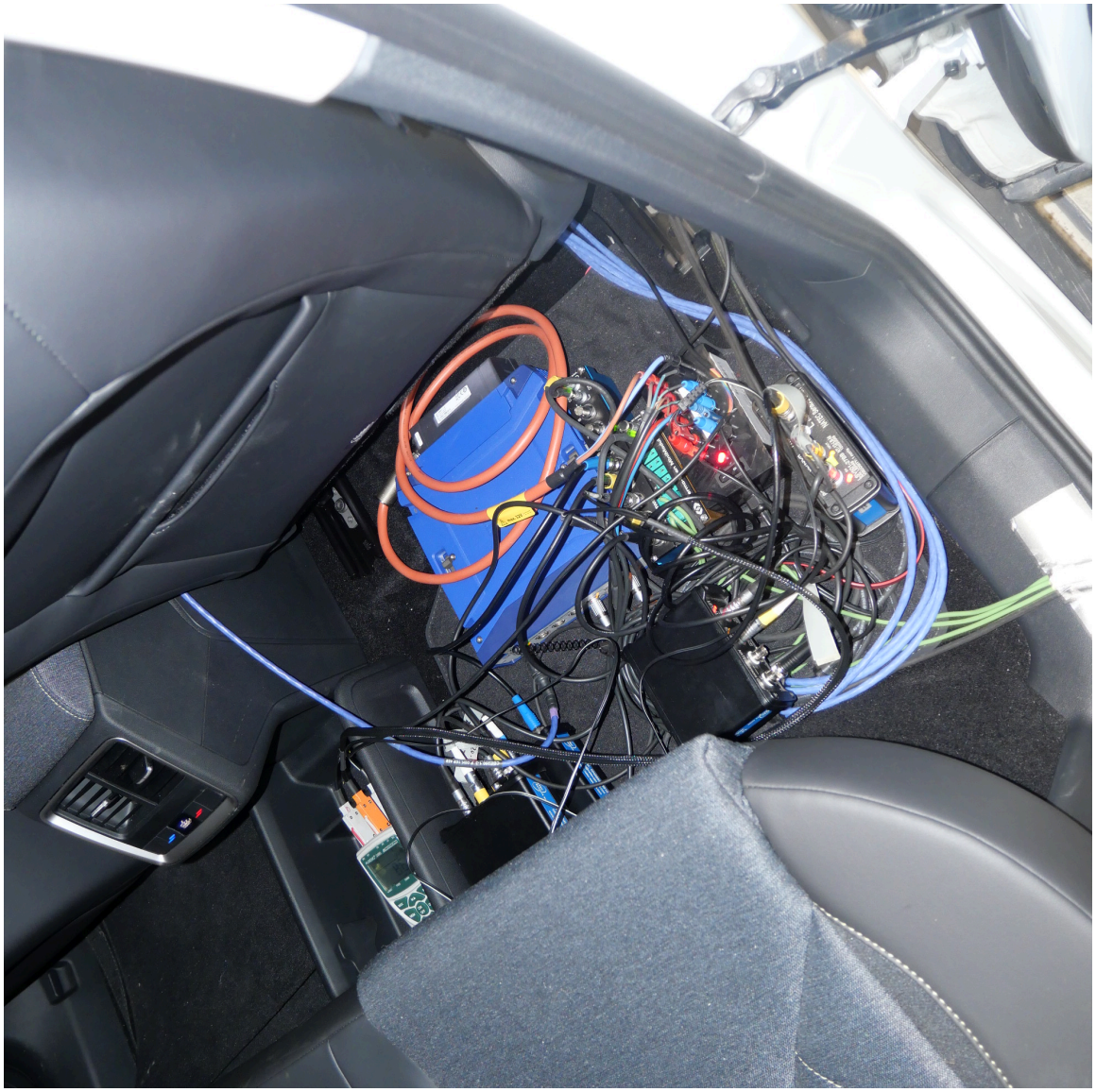


Figure 7. 3: ETAS hardware installed in the backseat of a test vehicle for data aggregation.

ATTACHMENT 5

Figure 7. 4: Top-down view of the ETAS hardware installed in the backseat of a test vehicle for data aggregation.

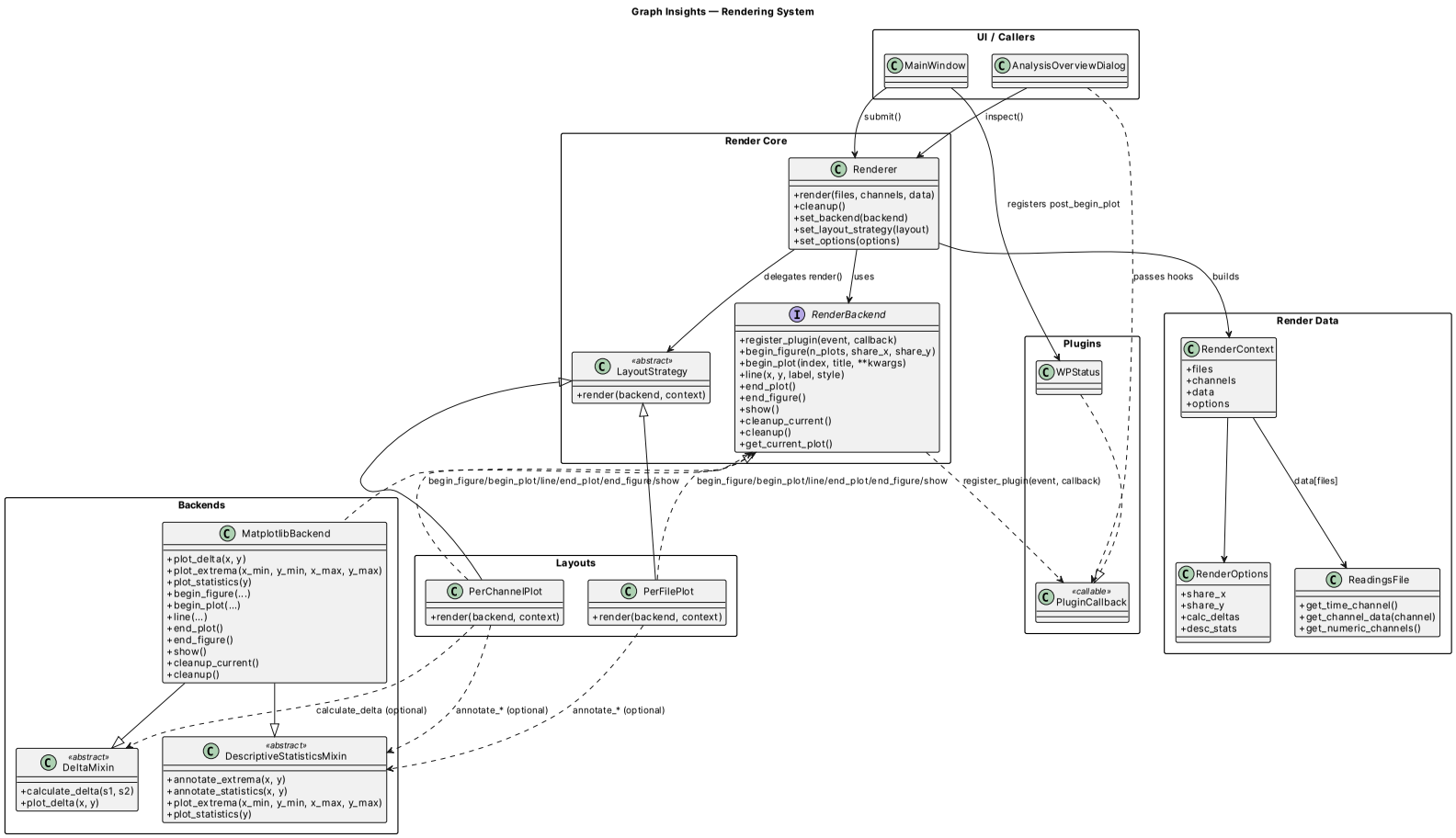


Figure 7. 5: Architecture of the rendering system, showing the core components and their interactions.

ATTACHMENT 7

For linguistic and research purposes, the use of Artificial Intelligence (AI) was leveraged throughout some parts of the thesis, more specifically the *GPT-5*.